

Aritmética de computadoras

Los dos aspectos fundamentales de la aritmética del computador son la forma de representar los números (el formato binario) y los algoritmos utilizados para realizar las operaciones aritméticas básicas (suma, resta, multiplicación y división). Enteros y flotantes.

Las cantidades en coma flotante se expresan como un número (mantisa) multiplicado por una constante (base) elevada a una potencia entera (exponente). Números muy grandes o muy chiquitos.

La unidad Aritmético-Lógica (ALU)

La ALU es la parte del computador que realiza realmente las operaciones aritméticas y lógicas con los datos. El resto de los elementos del computador (Unidad de control, registros, memorias, E/S) están principalmente para suministrar datos a la ALU, a fin de que esta los procese y para recuperar los resultados.

Una unidad aritmético-lógica, y en realidad todos los componentes electrónicos del computador, se basan en el uso de dispositivos lógicos digitales sencillos que pueden almacenar dígitos binarios y realizar operaciones lógicas booleanas elementales.

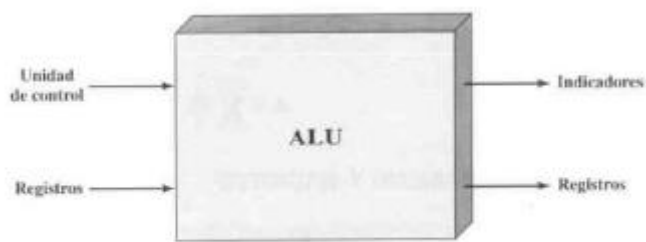


Figura 9.1. Entradas y salidas de la ALU.

Los datos se presentan a la ALU en registros, y en registros se almacenan los resultados de las operaciones producidos por la ALU. Estos registros son posiciones de memoria temporal internas al procesador que están conectados a la ALU. La ALU puede también activar indicadores (flags) como resultado de una operación.

Organización de los sistemas de computadora

Una computadora digital consiste en un sistema de procesadores interconectados, memorias y dispositivos de entrada/salida.

Procesadores

La CPU (Unidad central de procesamiento) es el “cerebro” de la computadora. Su función es ejecutar programas almacenados en la memoria principal buscando sus instrucciones y examinándolas para después ejecutarlas una tras otra. Los componentes están conectados por un **bus**, que es una colección de alambres paralelos para transmitir direcciones, datos y señales

de control. Los buses pueden ser externos a la CPU.

Unidad central de procesamiento (CPU)

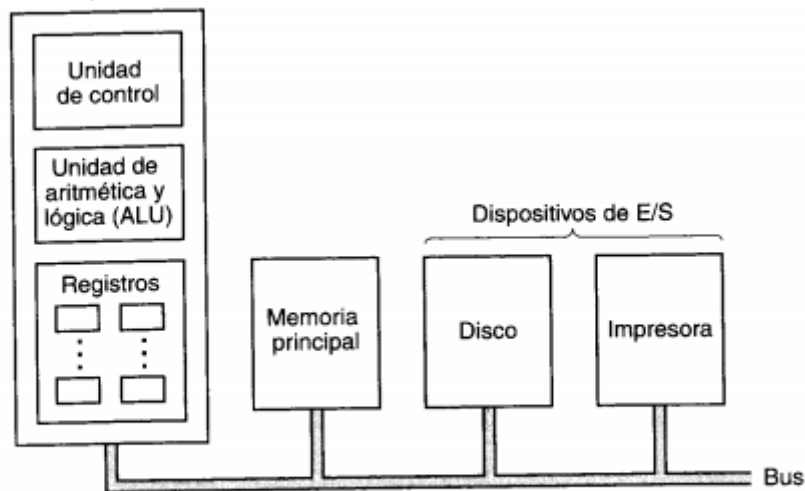


Figura 2-1. Organización de una computadora sencilla con una CPU y dos dispositivos de E/S.

La CPU se compone de varias partes. La unidad de control se encarga de buscar instrucciones de la memoria principal y determinar su tipo. La unidad de aritmética y lógica realiza operaciones como suma y AND booleano necesarias para ejecutar las instrucciones.

La CPU también contiene una memoria pequeña y de alta velocidad que sirve para almacenar resultados temporales y cierta información de control. Esta memoria se compone de varios registros, cada uno de los cuales tiene cierto tamaño y función. Por lo regular, todos los registros tienen el mismo tamaño. Cada registro puede contener un número, hasta algún máximo determinado por el tamaño del registro. Los registros pueden leerse y escribirse a alta velocidad porque están dentro del CPU.

El registro más importante es **ProgramCounter (PC)**, que apunta a la siguiente instrucción que debe buscarse para ejecutarse. Otro registro importante es el **IntruccionRegister (IR)** que contiene la instrucción que se esta ejecutando.

Organización de la CPU

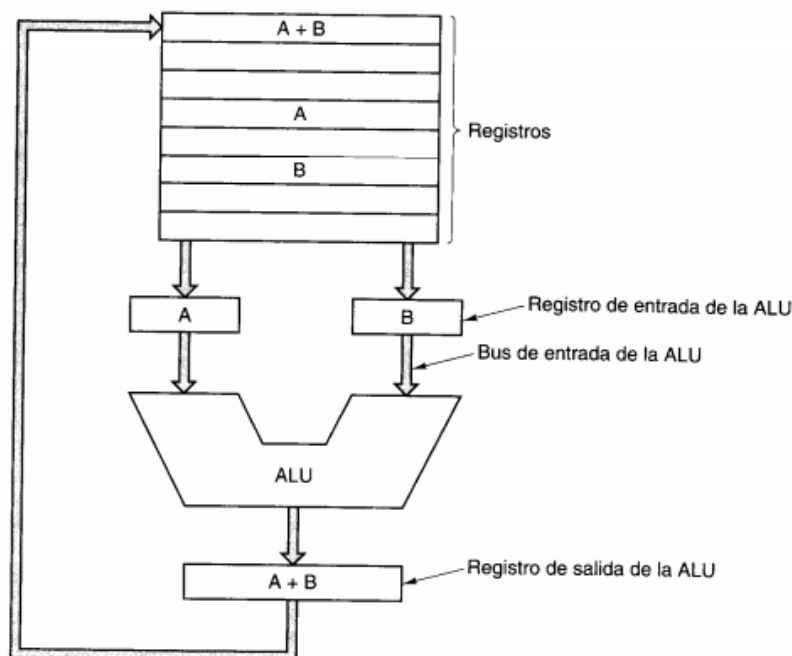


Figura 2-2. Camino de datos de una máquina von Neumann típica.

Esta parte se llama **camino de datos** y consiste en los registros (generalmente del 1 al 32), la **ALU** y varios buses que conectan los componentes. Los registros alimentan dos registros de entrada de la ALU, rotulados A y B en la figura. Estos registros contienen las entradas de la ALU mientras ésta está calculando.

La ALU hace su operación y produce un resultado en el registro de salida. El contenido de este registro de salida se envía a un registro, que posteriormente se escribe en la memoria, si se desea. No todos los diseños tienen los registros A,B y de salida.

Casi todas las instrucciones pueden dividirse en una de dos categorías

Registro-memoria: permiten buscar palabras de la memoria a los registros, donde pueden utilizarse como entradas de la ALU. Otras instrucciones permiten almacenar el contenido de un registro en la memoria

Registro-registro: Busca dos operando de los registros, los coloca en los registros de entrada de la ALU realiza alguna operación con ellos y coloca el resultado en uno de los registros.

El proceso de hacer pasar dos operando por la ALU y almacenar el resultado se llama **ciclo del camino de datos** y es el corazón de casi todas las CPU. En gran medida, este ciclo define lo que una maquina puede hacer. Cuando más rápido es el ciclo, más rápidamente opera la máquina.

Ejecución de instrucciones

La CPU ejecuta cada instrucción en una serie de pasos pequeños. A grandes rasgos, los pasos son los siguientes:

1. Buscar la siguiente instrucción de la memoria y colocarla en el registro de instrucciones
2. Modificar el contador de programa de modo que apunte a la siguiente instrucción.
3. Determinar el tipo de la instrucción que se trajo.
4. Si la instrucción utiliza una palabra de la memoria, determinar dónde está.
5. Buscar la palabra, si es necesario y colocarla en un registro de la CPU.
6. Ejecutar la instrucción.
- 7 Volver la paso 1 para comenzar a ejecutar la siguiente instrucción.

Esta sucesión de pasos se conoce como el ciclo de **búsqueda-decodificación-ejecucion**, y es fundamental para el funcionamiento de todas las computadoras.

Paralelismo en el nivel de instrucciones

Los arquitectos de computadoras se esfuerzan constantemente por mejorar el desempeño de las maquinas que diseñan. Una forma de hacerlo es aumentar la velocidad de reloj de los chips para que operen con mayor rapidez, pero para cada nuevo diseño existe un límite.

Paralelismo en el nivel de instrucciones: se aprovecha el paralelismo dentro de las instrucciones individuales para lograr que la maquina ejecute mas instrucciones por segundo.

Paralelismo en el nivel de procesador: múltiples CPU trabajan juntas en el mismo problema.

Filas de procesamiento (pipeline)

En lugar de dividir la ejecución de instrucciones en solo dos partes, a menudo se divide en muchas partes, cada una de las cuales se maneja con un componente de hardware dedicado, y todos estos componentes pueden operar en paralelo.

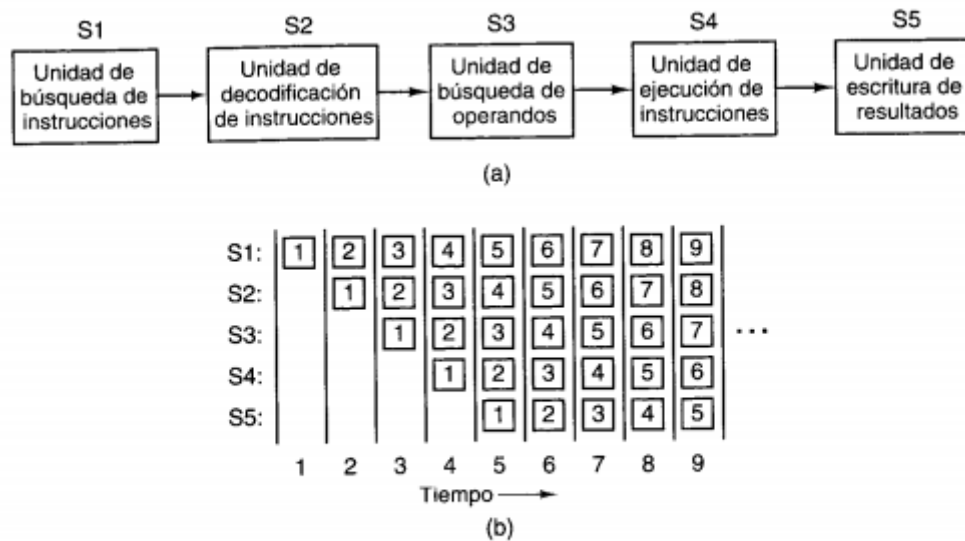


Figura 2-4. (a) Fila de procesamiento de cinco etapas. (b) Estado de cada etapa en función del tiempo. Se ilustran nueve ciclos de reloj.

Durante el ciclo de reloj 1, la etapa S1 está trabajando con la instrucción 1, buscándola en memoria. Durante el ciclo 2, la etapa S2 decodifica la instrucción 1, mientras la etapa S1 busca la instrucción 2... etc.

Arquitecturas superescalares

Si una fila de procesamiento es buena, entonces dos serán mejores.

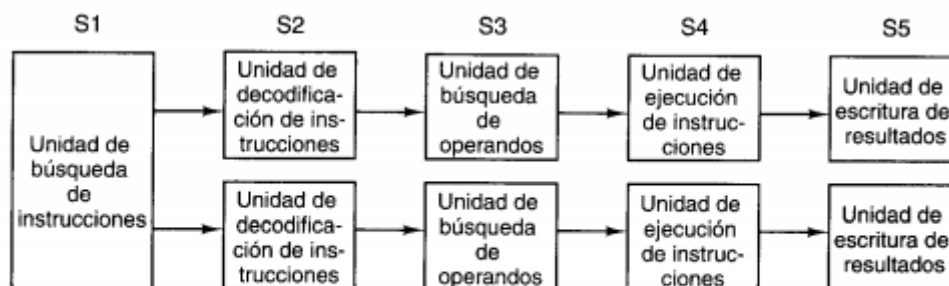
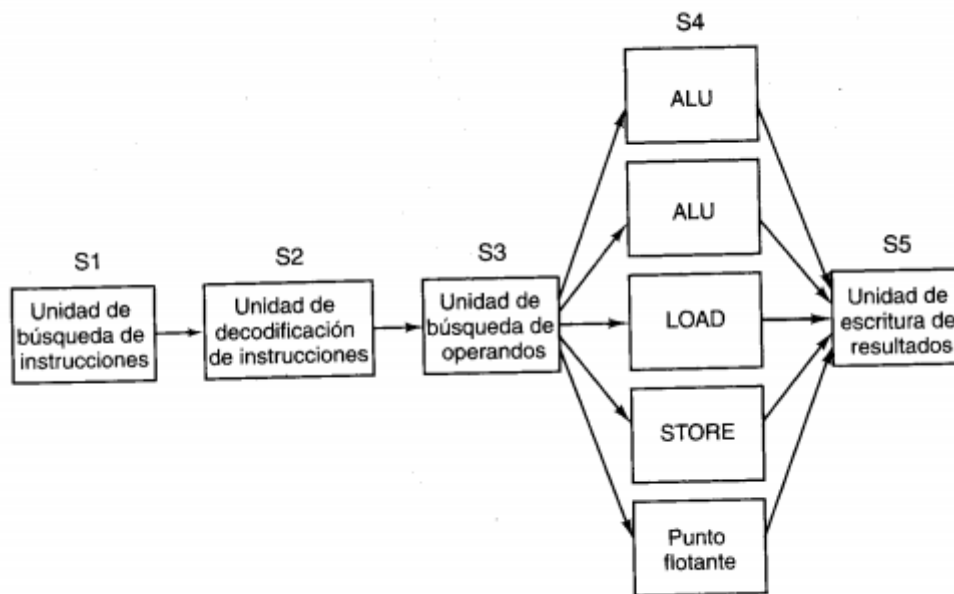


Figura 2-5. Filas de procesamiento dual de cinco etapas con unidad de búsqueda de instrucciones común.

Aquí una sola unidad de búsqueda de instrucciones trae pares de instrucciones y coloca cada una en su propia fila de procesamiento, que cuenta con su propia ALU para poder operar en paralelo. Esto requiere que las dos instrucciones no compitan por el uso de recursos y también que una no dependa del resultado de la otra. Al igual que con una sola fila de procesamiento, el compilador debe garantizar que se cumpla esta situación, o bien los conflictos se detectan y eliminan durante la ejecución utilizando hardware adicional.

Estrategia que acuñó el termino arquitectura superescalar

La idea básica es tener una sola fila de procesamiento pero proporcionarle varias unidades funcionales



El concepto de procesador superescalar lleva implícita la idea de que la etapa S3 puede emitir instrucciones con mucha mayor rapidez de la que la etapa S4 puede ejecutarlas. Si la etapa S3 emitiera una instrucción cada 10ns y todas las unidades funcionales pudieran efectuar su trabajo en 10 ns, solo una estaría ocupada en un momento dado, y la idea no representaría ninguna ventaja. En realidad, casi todas las unidades funcionales de la etapa 4 tardan mucho más que un ciclo de reloj en ejecutarse sobre todo las que acceden a la memoria o realizan aritmética de punto flotante. Como puede verse, es posible tener varios ALU en la S4.

Multiprocesadores

Un sistema con varias CPU que comparten una memoria común, como un grupo de personas en un salón que comparten un pizarrón común. Puesto que cada CPU puede leer o escribir en cualquier parte de la memoria, deben coordinarse(en software) para no estorbarse mutuamente.

Hay varios posible esquemas de implementación.

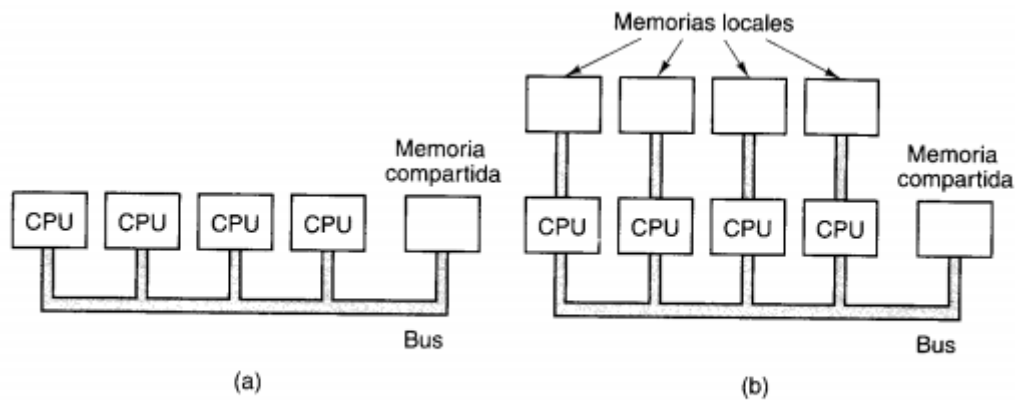


Figura 2-8. (a) Multiprocesador con un solo bus. (b) Multicomputadora con memorias locales.

Memoria primaria

La memoria es la parte de la computadora en la que se almacenan programas y datos.

Bits

La unidad básica de memoria es el dígito binario, llamado bit. Un bit puede contener un 0 o un 1; es la unidad más simple posible.

Direcciones de memoria

Las memorias consisten en varias **celdas**, cada una de las cuales puede almacenar un elemento de información. Cada celda tiene un número, su **dirección**, con el cual los programas pueden referirse a ella. Si una memoria tiene n celdas, tendrán direcciones de 0 a $n-1$. Todas las celdas de una memoria contienen el mismo número de bits. Si una celda consta de k bits, podrá contener cualquiera de las 2^k combinaciones de bits distintas.

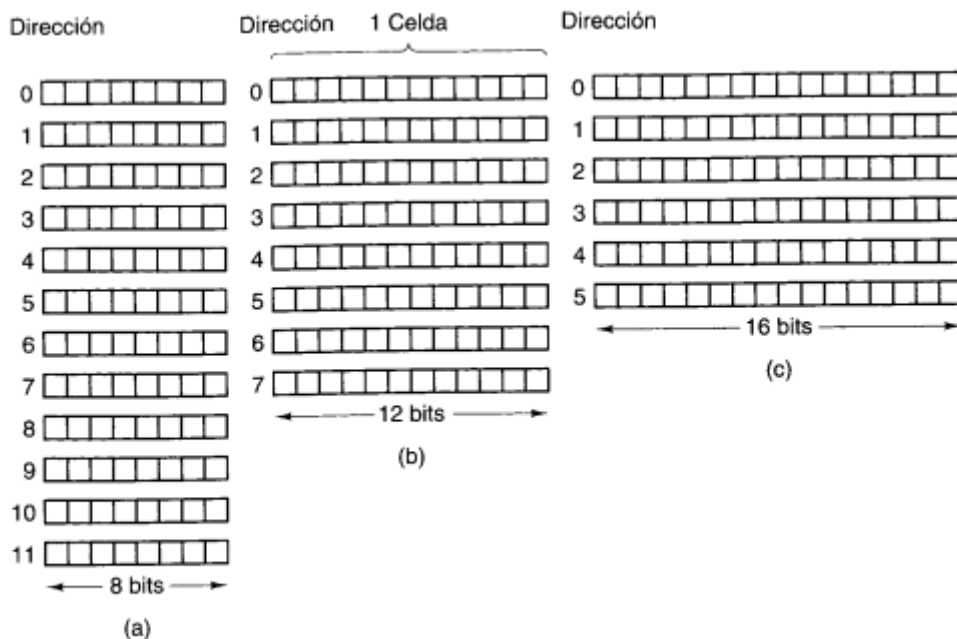


Figura 2-9. Tres formas de organizar una memoria de 96 bits.

El número de bits de la dirección determina el número máximo de celdas direccionables directamente en la memoria y es independiente del número de bits por celda. Una memoria que tiene 2^{12} celdas de 8 bits cada una, y una que tiene 2^{12} celdas de 64 bits cada una, necesitan ambas direcciones de 12 bits.

La importancia de la celda es que es la unidad direccionable mas pequeña. En años recientes casi todos los fabricantes de computadoras han adoptado como estándar una celda de 8 bits, que recibe el nombre de **byte**. Los byte se agrupan en **palabras**. Una computadora con palabras de 32 bits tiene 4 bytes/palabra, mientras que una con palabras de 64 bits tiene 8 bytes/palabra. La importancia de las palabras es que casi todas las instrucciones operan con palabras enteras.

Ordenamiento de bytes

Los bytes de una palabra pueden numerarse de izquierda a derecha o de derecha a izquierda.

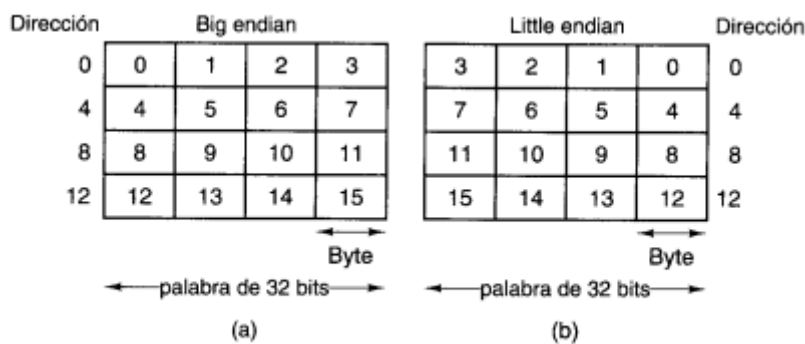


Figura 2-11. (a) Memoria big endian. (b) Memoria little endian.

El nivel de microprogramación

Tiene una función específica que es ejecutar intérpretes de otras máquinas virtuales.

Repaso: El trabajo del micro programador es escribir programas que controlen los registros, buses, unidades aritméticas y lógicas, memorias y otros componentes del hardware de las máquinas.

Registros: es un dispositivo capaz de almacenar información. El nivel de microprogramación siempre dispone de registros para guardar la información que se necesita en el procesamiento de la instrucción en curso de interpretación.

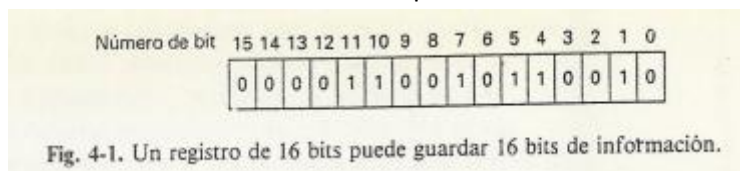
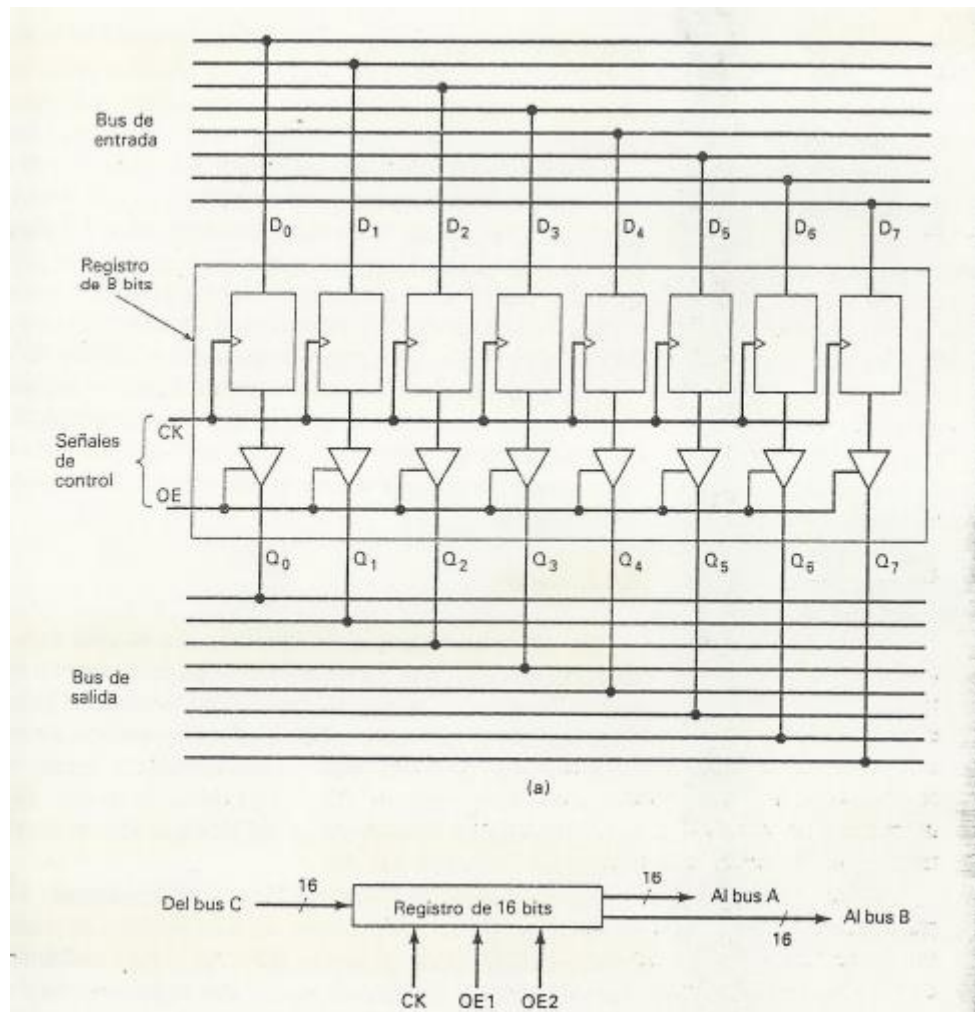


Fig. 4-1. Un registro de 16 bits puede guardar 16 bits de información.

Desde el punto de vista conceptual, los registros son lo mismo que la memoria principal; la diferencia estriba en que los registros están ubicados en el procesador mismo y, por lo tanto, se puede acceder a ellos en lectura y escritura más rápidamente que la memoria principal, la cual suele estar fuera del chip.

Buses: es un conjunto de alambres que se usan para transmitir señales en paralelo. Se emplean buses, pues la transmisión paralela de todos los bits a la vez, es mucho más rápida que la transmisión serial bit por bit.

Un bus puede ser unidireccional o bidireccional. Un bus unidireccional puede transferir información en un solo sentido; en cambio uno bidireccional puede transferirla en los dos sentidos, pero no simultáneamente. Un bus cuyos dispositivos tengan esta propiedad se llama bus **triestado**, porque cada línea puede estar a 0, a 1 o desconectada. Suele utilizarse cuando hay muchos dispositivos que pueden suministrar información a un bus.

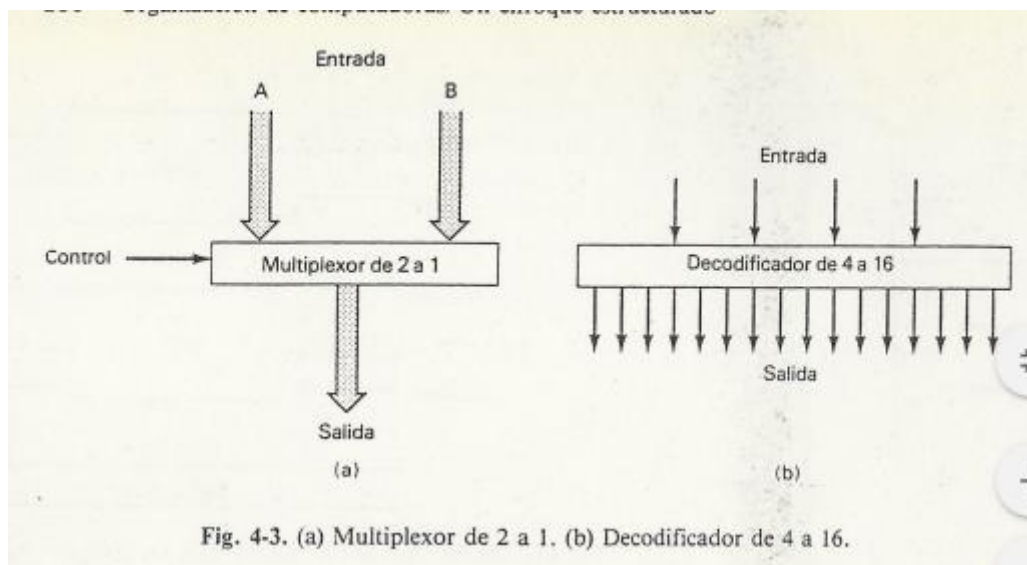


CK es un reloj, que realmente quiere decir "carga registro" y O E da el permiso de salida, ambas están conectadas a los flip-flops. Normalmente ambas señales se hallan en su estado de reposo, pero a veces pueden ser activadas, causando alguna acción.

Cuando CK están desactivada, el contenido del registro no es afectado por las señales del bus. En cambio, cuando se activa, el registro se carga con el contenido del bus de entrada. Cuando OE esta desactivada, el registro esta desconectado del bus de salida y desaparece respecto a los restantes registros conectados. Cuando OE esta activo, el contenido de registro pasa al bus de salida.

Multiplexores y decodificadores

Los circuitos que tienen una o más línea de entrada y que calculan uno o varios valores de salida determinados únicamente por las entradas actuales se llaman circuitos combinacionales. Dos de los más importantes son los multiplexores y los decodificadores.



Un **multiplexor** tiene 2^n entradas de datos (líneas individuales o buses), una salida de datos de la misma anchura que la entrada y una entrada de control de n bits que selecciona una de las entradas y la encamina a la salida.

El inverso del multiplexor es el **demultiplexor**, que encamina su única entrada a una de sus 2^n salidas, según el valor que tengan sus n líneas de control.

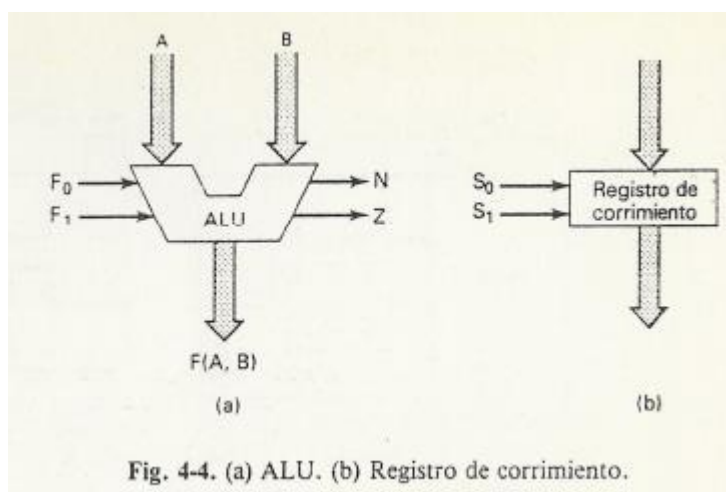
Un **decodificador**, tiene n líneas de entrada y 2^n de salida, numeradas desde 0 hasta $2^n - 1$. Si el número que hay en las líneas de entrada es k , entonces se pone a 1 la salida k , permaneciendo las demás en 0. Un decodificador tiene siempre una y una sola salida a 1, estando a 0 las restantes.

El inverso del decodificador es el **codificador**, que tiene 2^n entradas y n salidas. Solo puede haber una línea de entrada a 1 y su número, en binario, aparece en la salida.

Unidades aritméticas y lógicas y registros de corrimiento

Toda computadora necesita algún medio para hacer operaciones aritméticas.

La **ALU** tiene dos entradas y una salida de datos, pero además tiene algunas entradas y salidas de control.



Funciones de la ALU $\rightarrow A+B, A \text{ AND } B, A, \sim A$ (Están determinados por F0 y F1)

Una ALU también puede tener salidas de control. Salidas típicas son aquellas que se ponen a 1 cuando la salida de la ALU es negativa o cero, cuando hay acarreo del bit más significativo o cuando ha ocurrido un desbordamiento. Por ejemplo N indica si la salida es negativa y Z indica que es cero. El bit N es simplemente una copia del más significativo de salida, mientras que el bit Z es un NOR de todos los bits de salida.

Aunque algunas ALU puedan realizar también operaciones de desplazamiento, la mayoría de las veces es necesaria una unidad específica. Este circuito puede desplazar su entrada un bit a la izquierda o a la derecha.

Relojes

Los circuitos de las computadoras funcionan normalmente al ritmo de un reloj, dispositivo que emite una secuencia periódica de impulsos. Estos impulsos definen los ciclos de máquina. Durante cada ciclo tiene lugar alguna actividad básica, como la ejecución de una microinstrucción. A menudo es útil dividir los ciclos en subciclos, para que las partes de una microinstrucción puedan realizarse en un orden bien definido.

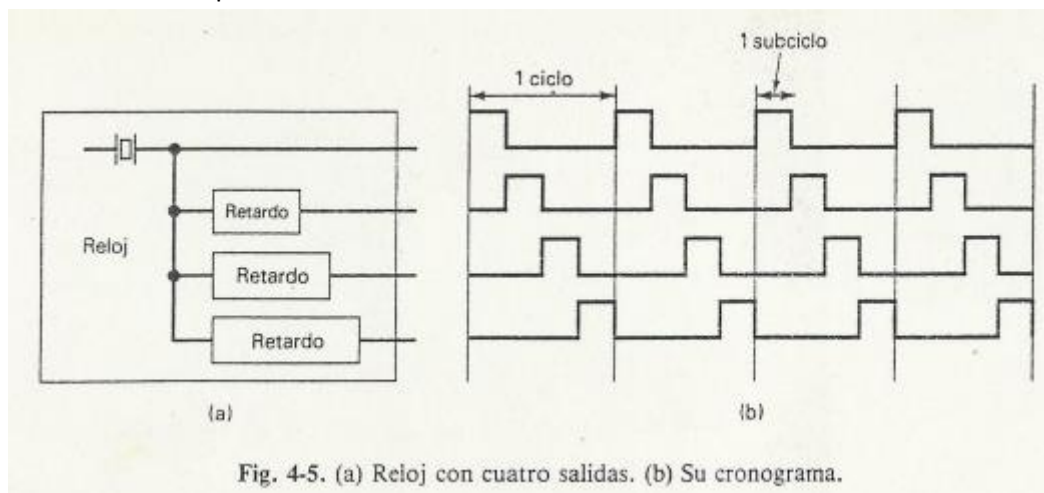
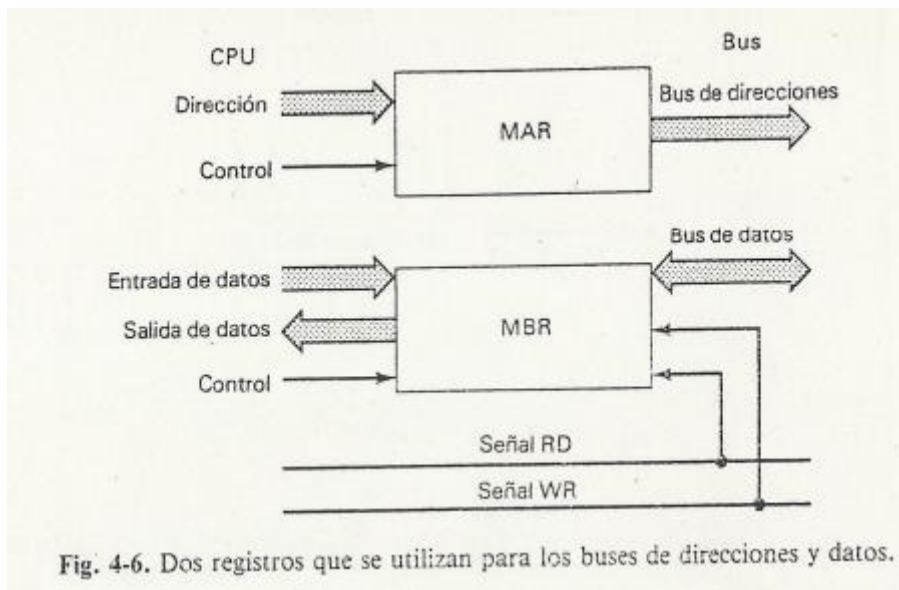


Fig. 4-5. (a) Reloj con cuatro salidas. (b) Su cronograma.

Memoria principal

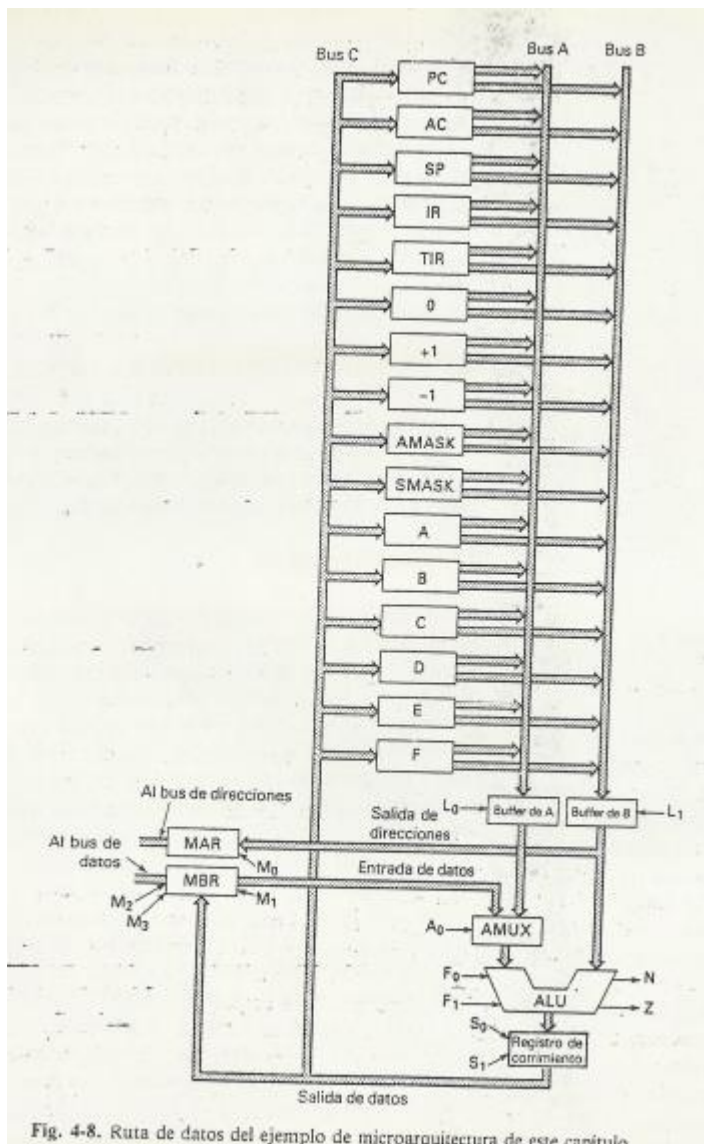
Los procesadores deben ser capaces de leer y escribir en memoria. La mayoría de las computadoras tiene un bus (conductor) de direcciones, otro de datos y otro de control para la comunicación entre la CPU y la memoria. Las lecturas se activan con la señal RD y las escrituras con la señal WR.

Un acceso a memoria es siempre considerablemente más largo que el que se necesita para ejecutar una sola microinstrucción. En consecuencia, el microprograma debe mantener los valores correctos en los buses de datos y direcciones durante varias microinstrucciones. Para simplificar esta tarea, a menudo es conveniente tener dos registros, el **MAR(registro de Direccion de Memoria)** y el **MBR (Registro de Intercambio de Memoria)**, a los que se conectan los buses de direcciones y de datos, respectivamente.



Una microarquitectura típica

La ruta de datos: es parte de la CPU que contiene a la ALU, sus entradas y sus salidas.



Ni el bus A ni el B están conectados directamente a la ALU, sino que hay un par de registros buffer (tampón) intermedios. Estos se necesitan porque la ALU es un circuito combinacional. La ALU necesita estos stops porque sin o cargaría varios datos al mismo tiempo xd. La carga de estos buffer está controlada por L0 y L1.

Para la comunicación con la memoria: El MAR puede cargarse a partir del registro B, en paralelo con una operación de la ALU. La línea M0 controla la carga del MAR. En las escrituras se puede cargar el MBR con la salida del registro de corrimiento, en paralelo con el almacenamiento en la memoria de anotaciones o en lugar de el M1 controla la carga

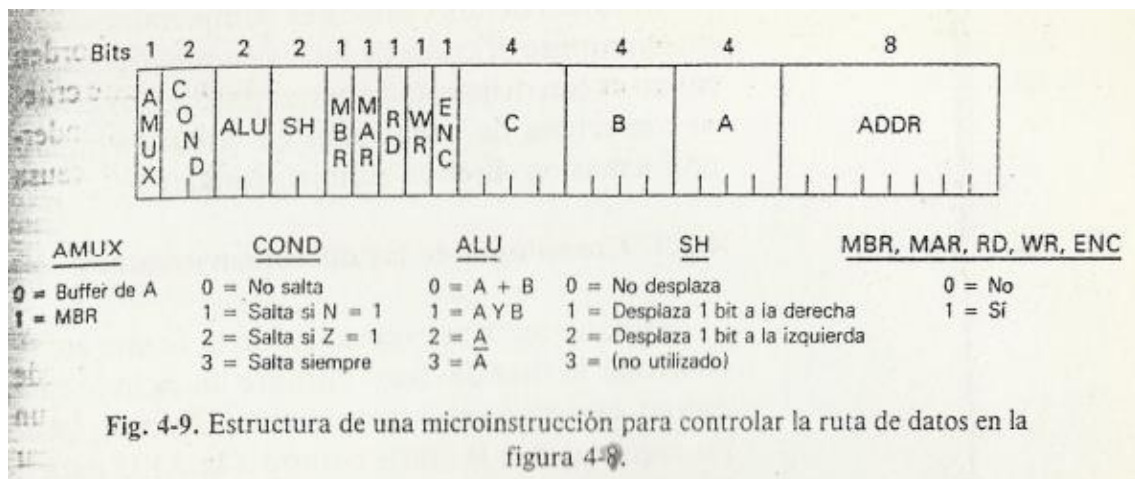
Del MBR a partir de la salida del registro de corrimiento M2 y M3 controlan las lecturas y escrituras de memoria.

Microinstrucciones

Para controlar la trayectoria de datos se requiere de 61 señales. Se pueden dividir en nueve grupos.

- *16 para controlar la carga del bus A a partir de registros internos.
- *16 para controlar la carga del bus B a partir de registros internos.
- *16 para controlar la carga de la memoria de anotaciones a partir del bus C.
- *2 para controlar los registros A y B. (L0 y L1)
- *2 para controlar la función de la ALU. (F0 y F1)
- *2 para controlar al registro de corrimiento. (S0 y S1)
- *4 para controlar el MAR y el MBR. (M0,M1,M2,M3)
- *2 para indicar una lectura o una escritura en memoria.
- *1 para controlar el Amux. (A0)

Con los valores de las 61 señales podemos realizar un ciclo de nuestra ruta de datos. Un ciclo consiste en vaciar los valores de los buses A y B, almacenarlos temporalmente en los registros A y B, en pasarlos a través de la ALU y el registro de corrimiento y en almacenar el resultado en la memoria interna y en el MBR o en ambos. Además, puede cargarse el MAR e iniciarse un ciclo de memoria.



Cronología de las microinstrucciones

Un ciclo básico de ALU consiste en cargar los registros A y B, darle tiempo a la ALU para realizar su trabajo y almacenar entonces el resultado.

Eventos claves durante cada uno de los subciclos:

1. Carga la siguiente microinstrucción a ejecutarse en un registro denominado **MIR**(Registro de MicroInstruccion)
2. Salida del contenido de los registros a los buses A y B, y su captura por los registros A y B.
3. Cuando las entradas de la ALU están estabilizadas, hay que dar tiempo a la ALU y al registro de corrimiento para que produzcan una salida estable y carga el MAR si es necesario
4. Ahora que la salida del registro de corrimiento está estabilizada, se almacena el bus C en la memoria de anotaciones y en el MBR si es necesario.

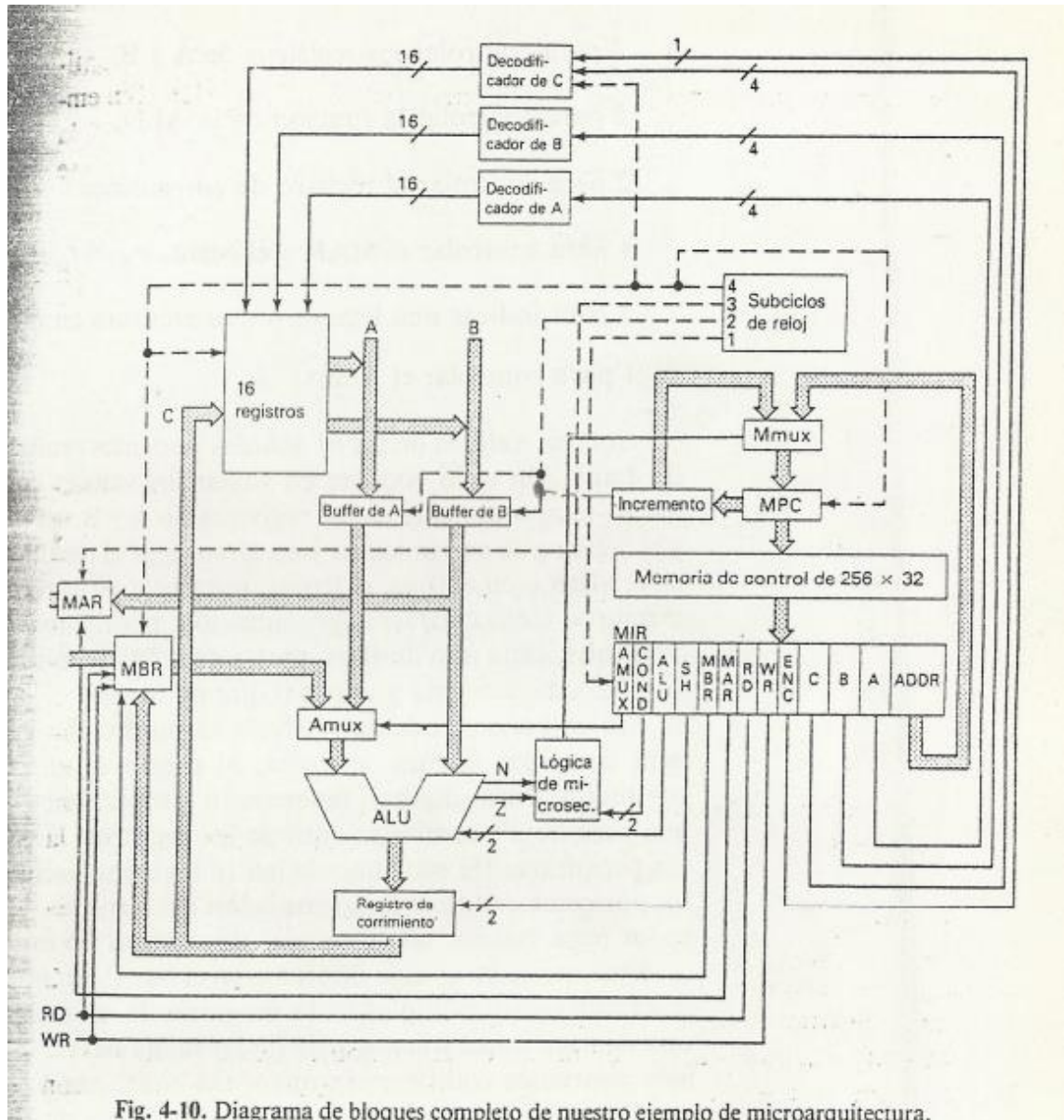


Fig. 4-10. Diagrama de bloques completo de nuestro ejemplo de microarquitectura.

La parte más voluminosa e importante de la porción de control de la maquina es la memoria de control. Es donde se guardan las microinstrucciones de escritura. Las microinstrucciones serán de 32 bits y el espacio de direcciones de microinstruccion constara de 256 palabras, la memoria de control ocupará un máximo de $256 \times 32 = 8192$ bits.

Como cualquier otra memoria, la de control necesita un MAR y un MBR. Llamaremos al MAR el **MPC (Contador de Microprogramas)**, porque su única función es señalar la siguiente instrucción que va a ejecutarse. El MBR será el **MIR** ya mencionado arriba.

1er subciclo: el MIR solamente se carga en este ciclo. Durante los otros tres subciclos no se altera, independientemente de lo que suceda al MPC.

2do subciclo: el MIR se encuentra estabilizado y sus campos empiezan a controlar la ruta de datos. El reloj activa los registros de A y de B durante este subciclo, proporcionando entradas estables a la ALU durante el resto del ciclo. Mientras los datos salen a los buses A y B, la unidad de "Incremento" de la sección de control acumula $MPC + 1$, en preparación de la carga siguiente.

3er subciclo: se les da a la ALU y al desplazador tiempo suficiente para que produzcan resultados válidos.

4to subciclo: el bus C se puede almacenar en la memoria de anotaciones y en el MBR, según los campos ENC y MBR. Solo se carga un registro de anotaciones si:

1. $ENC=1$
2. Es el subciclo 4.
3. El campo C selecciona el registro

El MBR también se carga durante este ciclo si $MBR=1$.

Secuenciamiento de las microinstrucciones

Lo único que queda por ver es como se elige la siguiente microinstrucción. Aunque la mayoría de las veces basta extraer la siguiente microinstrucción en secuencia, se necesita algún mecanismo que permita saltos condicionales en el microprograma para tomar decisiones. Por esta razón dotamos a cada microinstrucción de dos campos adicionales: ADDR, que es la dirección de un sucesor potencial de la microinstrucción en curso, y COND, que determina si la siguiente microinstrucción se extrae de $MPC + 1$ o de ADDR. Se ha tomado esta decisión porque los saltos condicionales son muy comunes en los microprogramas.

La elección de la siguiente microinstrucción la realiza la caja rotulada "Lógica de Microsecuenciamiento" durante el subciclo de 4, cuando las salidas de la ALU N y Z son válidas. La salida de esta caja controla el multiplexor M (Mmux), que encamina $MPC + 1$ o ADDR al MPC, que determinará cuál será la siguiente microinstrucción a extraer. Hemos proporcionado al microprogramador cuatro alternativas posibles ajustando COND como sigue:

0= No saltar; la siguiente microinstrucción se toma de $MPC + 1$

1= Saltar a ADDR si $N=1$.

2= Saltar a ADDR si $Z=1$.

3= Saltar a ADDR incondicionalmente

Una Macroarquitectura Típica

Llamaremos macroarquitectura a la arquitectura del nivel 2 o del 3. De modo similar, a las instrucciones del nivel 2 las llamaremos **macroinstrucciones** (ADD, MOVE, etc).

Pilas: consta de un bloque de memoria contigua, que contiene ciertos datos, y de un **apuntador a la pila** (SP), que dice dónde está la cima de ese bloque. La base de la pila esta en una dirección fija.

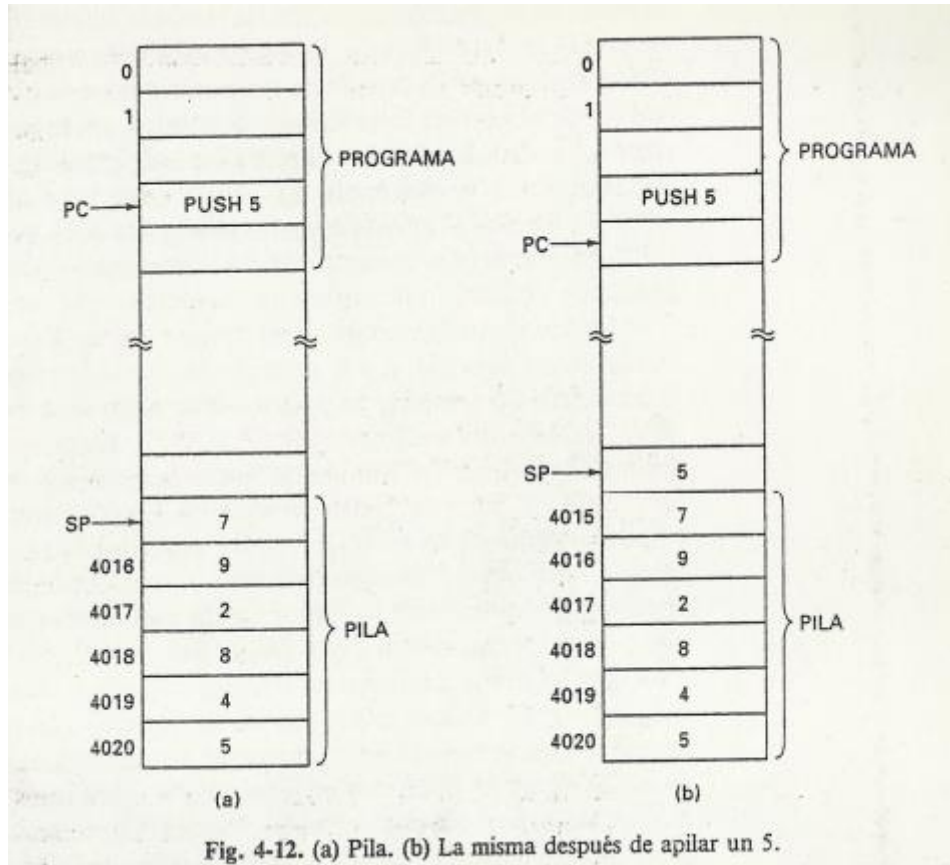


Fig. 4-12. (a) Pila. (b) La misma después de apilar un 5.

Se definen varias operaciones en las pilas. Las dos más importantes son **PUSH X** y **POP Y** (desapila Y), **PUSH** avanza el apuntador de pila (decrementándolo en el ejemplo) y luego pone X en la posición de memoria a la que ahora apunta SP. **PUSH** incrementa el tamaño de la pila en un elemento, **POP Y**, por lo contrario, reduce el tamaño de la pila guardándolo el último elemento en Y, eliminándolo de ella incrementando la dirección del apuntador de pila.

Otra operación que puede realizarse en una pila es avanzar el apuntador de pila sin apilar ningún dato. Esto suele hacerse cuando se entra en un procedimiento o función, par reservar espacio a las variables locales.

Java Virtual Machine

El nivel de MicroArquitectura

Este subconjunto contiene solo instrucciones para manejar enteros, por lo tanto lo hemos llamado **IJVM**

La JVM tiene algunas instrucciones relativamente complejas. El microprograma tiene un conjunto de variables, llamadas **estado** de la computadora, al que todas las funciones tienen acceso. Cada función modifica al menos alguna de las variables que constituyen el estado. Por ejemplo, el Contador de programa(PC) forma parte del estado; indica la localidad de memoria que contiene la siguiente función.

Las instrucciones de JVM son cortas y precisas. Cada instrucción tiene unos cuantos campos, por lo regular uno o dos, cada uno de los cuales tiene un propósito específico. El primer campo de cada instrucción es el **código de operación (opcode)** que identifica la instrucción como, por ejemplo, ADD o BRANCH. Muchas instrucciones tienen un campo adicional que especifica el operando. Por ejemplo las instrucciones que accedan una variable local necesitan un campo para indicar cual variable.

Este modelo de ejecución, llamado **ciclo de búsqueda-ejecución**, es útil en lo abstracto.

La trayectoria de datos

La **trayectoria de datos** es la parte del CPU que contiene la ALU, sus entradas y sus salidas.

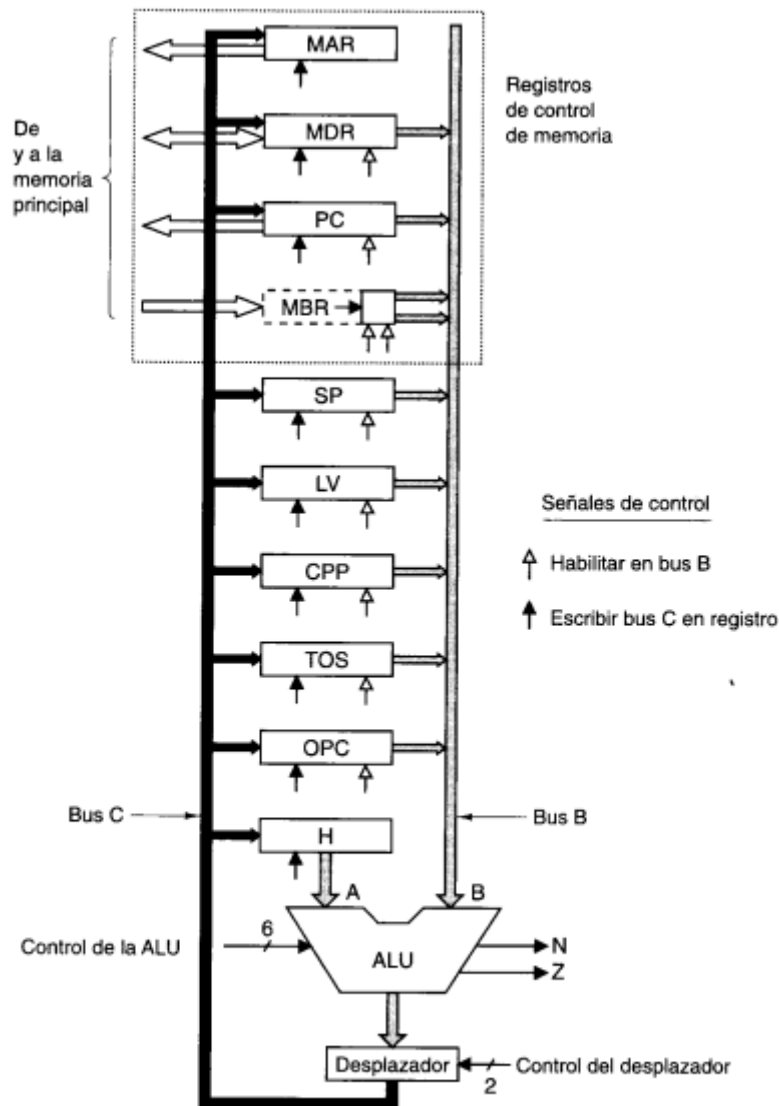


Figura 4-1. La trayectoria de datos de la microarquitectura que usaremos como ejemplo en este capítulo.

La función de la ALU esta determinada por seis líneas de control estas son:

F0 y f1: para determinar el funcionamiento de la ALU.

ENA y ENB para habilitar individualmente las salidas.

INVA para invertir la entrada izquierda

INC para forzar un acarreo de entrada en el bit de orden bajo, lo que en realidad suma 1 al resultado.

No todas las 64 combinaciones de las líneas de control de la ALU hacen algo útil.

F_0	F_1	ENA	ENB	INVA	INC	Función
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
0	1	0	1	1	0	$\bar{B}$
1	0	1	1	0	0	A + B
1	1	1	1	0	0	A + B + 1
1	1	1	1	0	1	A + 1
1	1	1	0	0	1	B + 1
1	1	0	1	0	1	B - A
1	1	1	1	1	1	B - 1
1	1	0	1	1	1	-A
1	1	1	0	1	1	-B
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
0	1	0	0	0	1	1
0	1	0	0	1	0	-1

Figura 4-2. Combinaciones útiles de señales de la ALU y la función que desempeñan.

Temporización de la trayectoria de datos

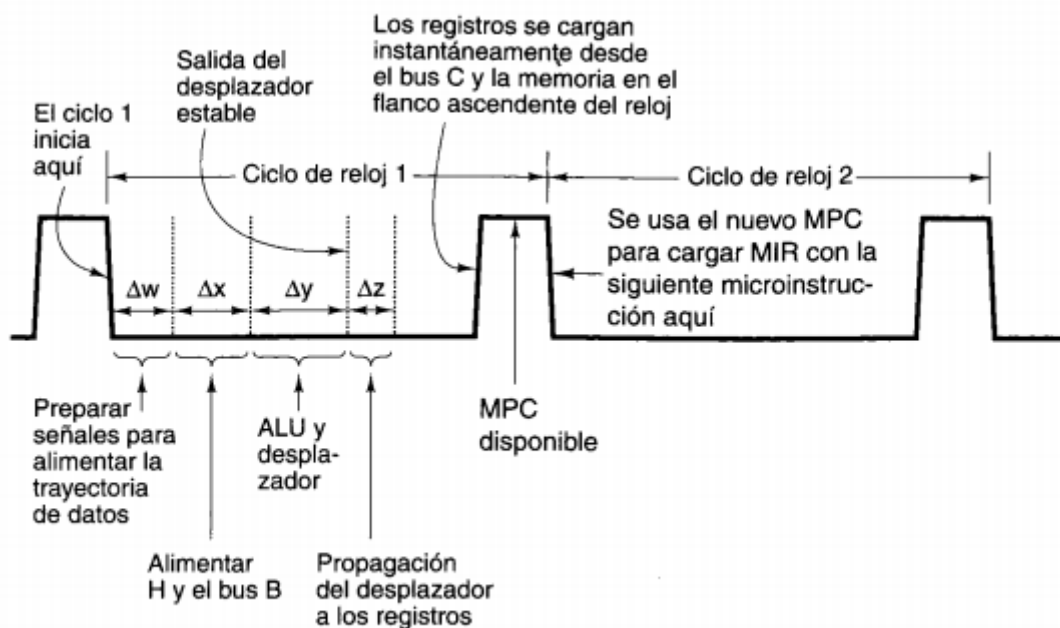


Figura 4-3. Diagrama de temporización de un ciclo de la trayectoria de datos.

Se produce una pulsación corta al principio de cada inicio de reloj. En el flanco descendente de la pulsación, ya están preparados los bits que alimentarán todas las compuertas. Esto tarda un tiempo finito conocido, Δw . Luego se selecciona el registro que se necesita en el bus B y su valor se pasa a ese bus. Se requiere un tiempo Δx para que el valor se estabilice. Luego la ALU y el desplazador comienzan a operar con datos válidos. Después de un tiempo Δy adicional, las salidas de la ALU y el desplazador se han estabilizado. Después de otro intervalo Δz , los

resultados se han propagado por el bus C hasta los registros, en los que pueden cargarse durante el flanco ascendente de la siguiente pulsación. La carga debe ser disparada por flanco y rápida, de modo que aun si algunos de los registros de entrada se modifican, los efectos no se harán sentir en el bus C sino hasta mucho después de haberse cargado todos los registros. También el flanco ascendente de la pulsación, el registro que alimenta al bus B dejade hacerlo, en preparación del siguiente ciclo. MPC, MIR y la memoria se mencionan en la figura; exploraremos sus papeles en breve.

Es importante darse cuenta de que aunque no hay elementos de almacenamiento en la trayectoria de datos, hay un tiempo de propagación finito a través suyo. Modificar el valor que está en el bus B no hace que el bus C cambie sino hasta después de un tiempo finito. Por consiguiente, aun si una operación de almacenamiento modifica uno de los registros de entrada, el valor ya estará guardado a salvo en el registro mucho antes de que el valor (ahora incorrecto) que se está colocando en el bus B (o en H) pueda llegar a la ALU.

1. Se preparan las señales de control (Δw).
2. Los registros se cargan en el bus B (Δx).
3. La ALU y el desplazador operan (Δy).
4. Los resultados se propagan por el bus C de regreso a los registros (Δz).

Operación de la memoria

La maquina tiene dos diferentes formas de comunicarse con la memoria: un puerto de memoria de 32 bits direccionable por palabra y un puerto de memoria de 8 bits direccionable por byte. El puerto de 32 bits está bajo el control de dos registros, **MAR** (Registro de dirección de memoria) y **MDR** (registro de datos de memoria). El puerto de ocho bits esta bajo el control de un registro, PC, que lee un byte y lo coloca en los ocho bits de orden bajo de **MBR**. Este puerto solo puede leer datos de la memoria; no puede escribir datos en la memoria.

Cada uno de los registros operan con una o dos **señales de control**. Una **flecha blanca** bajo un registro indica una señal de control que habilita la colocación de la salida del registro en el bus B. Una **flecha negra** indica una señal de control que escribe (es decir, carga) el registro a partir del bus C. Para iniciar una lectura o escritura de memoria, es preciso cargar los registros de memoria apropiados, y luego emitir una señal de lectura o escritura a la memoria.

MAR contiene direcciones de palabra, de modo que los valores 0,1,2,etc... se refieren a palabras consecutivas.

PC contiene direcciones de byte, de modo que los valores 0,1,2,etc... se refieren a bytes consecutivos.

EJEMPLO: Colocar un 2 en PC e iniciar una lectura de memoria hara que se lea el byte 2 de la memoria y se coloque en los 8 bits de orden bajo de MBR. Colocar un 2 en MAR e iniciar una lectura de memoria hara que se lean los byte 8-11 (o sea, la palabra 2) de la memoria y se coloquen en MDR.

Esta diferencia de funcionalidad es necesaria poque MAR y PC se usaran para hacer referencia a dos partes diferentes de la memoria. La combinación MAR/MDR se usa apra leer y escribir palabras de datos en el nivel ISA, y la combinación PC/MBR sirve para leer el programa

ejecutable en el nivel ISA, que consiste en un flujo de bytes. Todos los demás registros que contienen direcciones emplean direcciones de palabra, como MAR.

Los datos leídos de la memoria a través del puerto de memoria de ocho bits se devuelven en MBR, un registro de 8 bits. MBR puede copiarse en el bus B de una de dos formas: sin signo o con signo. Cuando se necesita un valor sin signo, la palabra de 32 bits se coloca en el bus B contiene el valor de MBR en los 8 bits de orden bajo y ceros en los 24 bits superiores. La otra opción para convertir el MBR de 8 bits en una palabra de 32 bits es tratarlo como un valor con signo entre -128 y +127 y usar este valor para generar una palabra de 32 bits que tenga el mismo valor numérico. Esta conversión se efectúa copiando el bit de signo de MBR en las 24 posiciones de bit superiores del bus B, este proceso se llama **extensión del signo**.

La decisión de convertir el MBR de 8 bits a un valor de 32 bits con o sin signo en el bus B depende de cuál de las dos líneas de control (flechas blancas debajo de MBR) está habilitada.

Microinstrucciones

Para controlar la trayectoria de datos necesitamos 29 señales. Estas se dividen en 5 grupos funcionales

9 señales para controlar la escritura de datos del bus C en registros

9 señales para controlar la habilitación de registros en el bus B para introducción en la ALU.

8 señales para controlar las funciones de la ALU y el desplazador

2 señales para indicar lectura/escritura de memoria via MAR/MDR.

1 señal para indicar obtención de memoria via PC/MBR.

Los valores de estas 29 señales de control especifican las operaciones durante un ciclo de la trayectoria de datos. Un ciclo consiste en colocar valores de los registros en el bus B, propagar las señales a través de la ALU y el desplazador, alimentarlas al bus C y por ultimo escribir los resultados en el registro o registros apropiados. Si una señal de lectura de datos de memoria está habilitada, la operación de memoria se inicia al final del ciclo de la trayectoria de datos, una vez que se ha cargado MAR. Los datos de memoria están disponibles hasta el final de siguiente ciclo en MBR o MDR y se usa el ciclo que sigue. En otras palabras, una lectura de memoria por cualquiera de los puertos, iniciada al final del ciclo k, proporciona datos que no se usan en el ciclo k+1, sino hasta el ciclo k+2 o uno posterior.

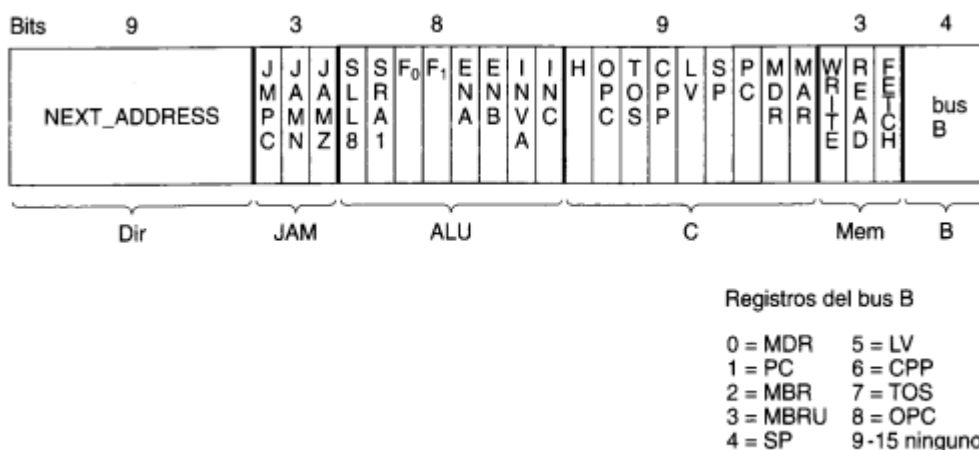


Figura 4-5. El formato de microinstrucciones para el Mic-1.

DIR -Contiene la dirección de una microinstrucción que podría ser la siguiente.

JAM -Determina como se selecciona la siguiente microinstrucción.

ALU -Funciones de la ALU y el desplazador.

C -Selecciona cuales registros se escriben desde el bus C

Mem -Funciones de memoria.

B -Selecciona la fuente del bus B; se codifica como se muestra.

Control de microinstrucciones: el mic-1

Esto lo determina un **secuenciador** que se encarga de recorrer la secuencia de operaciones necesarias para ejecutar una sola instrucción ISA.

El secuenciador debe producir dos tipos de información en cada ciclo:

1. El estado de cada señal de control en el sistema.
2. La dirección de la microinstrucción que debe ejecutarse a continuación.

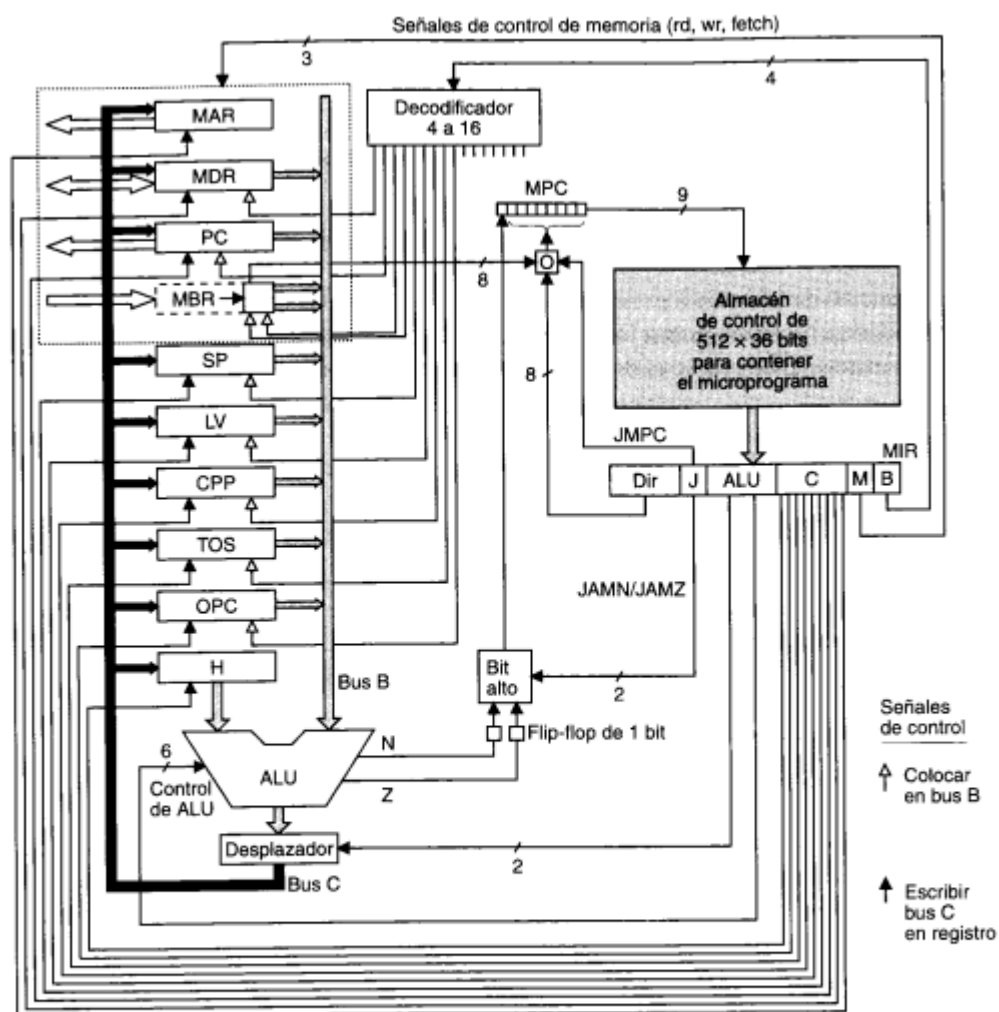


Figura 4-6. Diagrama de bloques completo de nuestra microarquitectura de ejemplo, Mic-1.

El elemento más grande e importante de la porción de control de la maquina es una memoria llamada **almacén de control**. Es recomendable ver este “almacén” como una memoria que contiene todo el microprograma, aunque a veces se implementa como un conjunto de compuertas lógicas. En lo funcional, el almacén de control no es más que una memoria que contiene microinstrucciones en lugar de instrucciones ISA.

Es importante notar que el almacén de control es muy diferente de la memoria principal: las instrucciones de la memoria principal tienen la propiedad de que se ejecutan en orden según su dirección; las microinstrucciones no. Los microprogramas necesitan mayor flexibilidad (porque las secuencias de microinstrucciones tienden a ser cortas), así que generalmente no tienen esta propiedad. En lugar de esto, cada microinstrucción especifica explícitamente su sucesora.

Puesto que el almacén de control es funcionalmente una memoria (solo de lectura) necesita su propio registro de direcciones de memoria y su propio registro de datos de memoria. Llamaremos al registro de dirección de memoria del almacén de control MPC. El registro de datos de memoria se llama MIR. Su función es almacenar la microinstrucción actual, cuyos bits alimentan las señales de control que operan la trayectoria de datos.

El registro MIR contiene los mismos seis grupos. Los grupos DIR y J controlan la selección de la siguiente microinstrucción. El grupo ALU contiene 8 bits que seleccionan la función de la ALU y controlan el desplazador. Los bits C hacen que registros individuales carguen la salida de la ALU desde el bus C. Los bits M controlan las operaciones de memoria. Por último, los cuatro bits finales controlan el decodificador que determina que se coloca en el bus B.

Una vez la microinstrucción esta en MIR, las diversas señales se propagan a la trayectoria de datos. Se coloca un registro en el bus B, la ALU sabe que instrucción debe ejecutar y ocurren muchas actividades mas. Este es el segundo subciclo. Después del intervalo $\Delta w + \Delta x$ a partir del principio del ciclo, las entradas de la ALU se han estabilizado.

Después de transcurrir un tiempo Δy adicional, todo se ha calmado otra vez y las salidas de la ALU, N, Z y el desplazador están estables. Luego, los valores de N y Z se guardan en un par de flip-flops de un bit. Estos bits, al igual que todos los registros que se cargan a partir del bus C y la memoria, se guardan en el flanco ascendente del reloj, cerca del final del ciclo de la trayectoria de datos. La salida de la ALU no se almacena, solo se alimenta al desplazador. La actividad de la ALU y el desplazador ocurre durante el subciclo 3

Después de un intervalo adicional, Δz , la salida del desplazador ha llegado a los registros por el bus C. Luego los registros pueden cargarse cerca del final del ciclo. El subciclo 4 consiste en el cargado de los registros y los flip-flops N y Z; terminan un poco después del flanco ascendente del reloj, cuando todos los resultados se han guardado y los resultados de las operaciones de memoria anteriores están disponibles, y se ha cargado el MPC. Este proceso continua hasta que alguien apaga la máquina.

Al mismo tiempo que controla la trayectoria de datos, el microprograma tiene que determinar que microinstrucción ejecutara a continuación, porque no se ejecutan en el orden en que aparecen en el almacén de control. El calculo de la dirección de la siguiente microinstrucción

se inicia una vez que MIR se ha cargado y estabilizado. Primero se copia el campo NEXT_ADDRESS de 9 bits en MPC. Mientras se esta efectuando este copiado, se inspecciona el campo JAM. Si tiene el valor 000, no se hace nada mas; una vez que concluye el copiado de NEXT_ADDRESS, MPC apunta a la siguiente microinstrucción.

Si uno o mas de los bits de JAM son 1, hay que trabajar mas. Si JAMN esta activado, el flip-flop N de un bit se hace OR con el bit de orden alto de MPC. Del mismo modo, si JAMZ esta establecido, el flip-flop Z de un bit realiza la operación OR. Si ambos están establecidos, se realiza la operación OR de ellos. Se necesitan los flip-flops N y Z porque después del flanco ascendentes del reloj (mientras el reloj esta alto), el B bus ya no esta siendo controlado, y no puede suponerse salidas de la ALU sigan siendo correctas. Guardar las banderas de estado de la ALU en N y Z hace que los valores correctos se estabilicen y estén disponibles para el calculo de MPC, sin importar que este ocurriendo en torno de la ALU.

Bit alto= (JAMZ AND Z) OR (JAMN AND N) OR NEXT_ADDRESS[8]

1. El valor de NEXT_ADDRESS
2. El valor de NEXT_ADDRESS después de que se realiza el OR del bit de orden alto con 1

No existe otras posibilidades. Si el bit de orden alto de NEXT_ADDRESS era 1, no tiene sentido usar JAMN o JAMZ

EJEMPLO: NEXT_ADDRESS = 0x92 y JAMZ=1 por lo tanto la siguiente dirección de la microinstrucción depende del bit Z almacenado en la operación ALU previa. Si el bit Z es 0, la siguiente microinstrucción provendrá de 0x92. Si el bit Z es 1, la siguiente microinstrucción provendrá de 0x192.

El tercer bit de campo JAM es JMP. Si esta activo, se efectúa la operación OR bit por bit de los ocho bits de MBR con los 8 bits de orden bajo del campo NEXT_ADDRESS de la microinstrucción en curso. El resultado se envía a MPC.

La capacidad de calcular el OR de MBR con NEXT_ADDRESS y almacenar el resultado en MPC hace posible una implementación eficiente de una ramificación (salto) de multiples vías. Es posible especificar cualquiera de las 256 direcciones, determinada exclusivamente por los bits presentes en MBR.

COMO 5to RESUMEN DE COMO FUNCIONA EL FUCKING CLOCK Y LA CONCHA DE SU MADRE

Durante el subciclo 1, iniciado por el flanco descendente del reloj, MIR se carga con la dirección que actualmente está en MPC. Durante el subciclo 2, las señales de MIR se propagan hacia afuera y el bus B se carga a partir del registro seleccionado. Durante el subciclo 3 la ALU y el desplazador operan y producen un resultado estable. Durante el subciclo 4, el bus C, los buses de memoria y los valores de la ALU se estabilizan. En el flanco ascendente del reloj los registros se cargan a partir del bus C, los flip-flops N y Z se cargan, y MBR y MDR obtienen sus resultados de la operación de memoria que se inició final del ciclo de trayectoria de datos anterior. Tan pronto como está disponible MBR, MPC se carga en preparación para la siguiente microinstrucción. Así MPC obtiene su valor en algún momento durante la parte media del

intervalo en que el reloj esta alto pero después de que MBR/MDR están listos. Esto podría ser disparado por nivel o disparador por flanco con un retraso fijo después del flanco ascendente del reloj. Lo único que importa es que MPC no se cargue sino hasta que los registros de los que depende (MBR,N y Z) estén listos. Tan pronto como el reloj baja, MPC puede direccionar el almacén de control y se puede iniciar un ciclo nuevo.

RESUMEN DEL RESUMEN DEL RESUMEN DEL RESUMEN:

Especifica que se coloca en el bus B, que deben hacer la ALU y el desplazador, donde debe almacenarse el valor del bus C y, por último, que valor debe tener MPC a continuación.

Macroarquitectura java

Pilas java

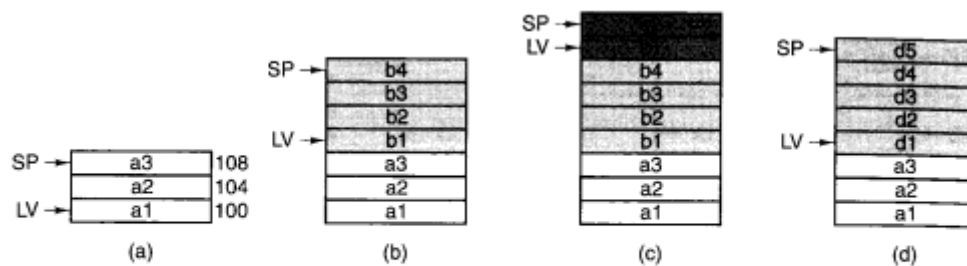


Figura 4-8. Uso de una pila para almacenar variables locales. (a) Mientras A está activo. (b) Después de que A llama a B. (c) Después de que B llama a C. (d) Después de que C y B regresan y A llama a D.

Para la ejecución de métodos, procedimientos...

Se reserva un área de memoria, llamada pila, para variables, pero las variables individuales no reciben direcciones absolutas dentro de la pila. En vez de ello se ajusta a un registro, digamos LV(Local variable) de modo que apunte a la base de las variables locales del procedimiento en curso. Otro registro, SP (Stack Pointer), apunta a la palabra más alta de las variables locales de A. La estructura de datos entre LV y SP (que incluye las dos palabras a las que apunta) es el **marco de variables locales**.

Si dentro del procedimiento se llama a otro procedimiento las próximas variables locales se almacenan arriba del procedimiento anterior, se ajusta LV de modo que apunte a las variables locales del segundo procedimiento. Podemos hacer referencias a las variables locales del segundo procedimiento dando su distancia respecto a LV. De misma manera si el segundo procedimiento llama a un tercer procedimiento.

Supongamos que terminamos con el tercer procedimiento la pila vuelve al estado que estaba en el segundo procedimiento. De igual modo cuando se termina el segundo procedimiento. En todas las condiciones LV apunta a la base del marco de la pila del procedimiento que está activo, y SP apunta al tope del marco de la pila. Cuando un procedimiento regresa, se libera la memoria que usaban sus variables locales.

Las pilas tienen otro uso pueden servir para retener operando durante el cálculo de una expresión aritmética. Cuando se usa este modo, la pila se llama **pila de operandos**.

EJEMPLO $a1 = a2 + a3$.

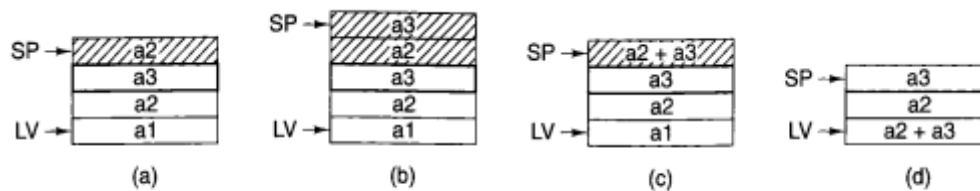


Figura 4-9. Uso de una pila de operandos para realizar un cálculo aritmético.

Se mete $a2$ y $a3$ en la pila y ahora puede efectuarse el cálculo ejecutando una instrucción que se obtiene dos palabras de la pila, las suma y mete el resultado otra vez a la pila. Por último, saca la variable que está en la localidad más alta de la pila y se almacena en la variable local $a1$.

Los marcos de variables locales y las pilas de operando se pueden entremezclar

EJEMPLO: al calcular $x^2 + f(x)$.

El modelo de memoria JVM

Basicamente consiste en una memoria que se puede ver de dos maneras: como una formación de 4,294,967,296 bytes (4GB) o como una formación de 1,073,741,824 palabras, cada una de 4 bytes. La MVJ no permite que el nivel ISA vea directamente las direcciones, pero hay varias direcciones implícitas que sirven de base para un apuntador. Las instrucciones de la JVM solo puede acceder a la memoria sumando índices a tales apuntadores. En cualquier momento están definidas las siguientes áreas de memoria:

1. La reserva de constantes. Un programa JVM no puede escribir en esta área que consta de constantes, cadenas y apuntadores a otras áreas de memoria a las que se hace referencia. Se carga cuando el programa se almacena en la memoria y no se modifica posteriormente. Existe un registro implícito, CPP, que contiene la dirección de la primera palabra de la reserva de constantes.
2. El marco de variables locales. Para cada invocación de un método, se asigna un área de almacenar variables durante la vigencia de la invocación. Esta área se llama **marco de variables locales**. Al principio de este marco residen los parámetros (también llamados argumentos) con que se invocó el método. El marco de variables locales no incluye la pila de operandos, que es aparte. Sin embargo, por razones de eficiencia, nuestra implementación coloca la pila de operandos inmediatamente arriba del marco de variables locales. Existe un registro implícito que contiene la dirección de la primera posición del marco de variables locales. Llamaremos a este registro LV. Los parámetros pasan en el momento de invocarse el método y se almacenan al principio de variables locales.
3. La pila de operandos. Se garantiza que el marco de la pila no excederá cierto tamaño, calculado con anticipación por el compilador de Java. El espacio de la pila de operandos se asigna directamente arriba del marco de variables locales. En

nuestra implementación es conveniente pensar en la pila de operandos como parte del marco de variables locales. En todo caso, existe un registro implícito que contiene la dirección de la palabra superior de la pila. Observe que, a diferencia de CPP y LV, este apuntador, SP, cambia durante la ejecución del método a medida que se meten operandos en la pila o se sacan de ella

4. El área de métodos. Por último, hay una región de la memoria que contiene el programa y que se llama área de “texto” en el caso de un proceso UNIX. Existe un registro implícito que contiene la dirección de la instrucción que se traerá a continuación. Este apuntador es el Contador de Programa o PC. A diferencia de las demás regiones de memoria, el área de métodos se trata como una formación de bytes.

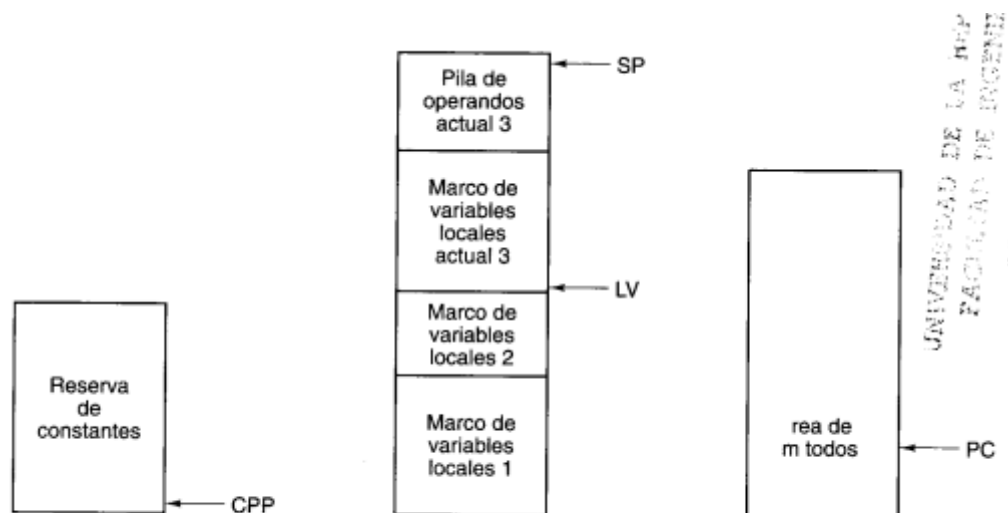


Figura 4-10. Las distintas partes de la memoria JVM.

Los registros CPP, LV y AP son apuntadores a palabras, no a bytes, y las distancias se miden en palabras

El conjunto de instrucciones JVM

Cada instrucción consiste en un código de operación y a veces un operando, como una distancia en la memoria o una constante. La primera columna da la codificación hexadecimal de la instrucción. La segunda da su representación mnemotécnica en lenguaje ensamblador. La tercera da un breve descripción de su efecto.

Hex	Mnemónico	Significado
0x10	BIPUSH <i>byte</i>	Almacenar byte en la pila
0x59	DUP	Copiar la palabra en el tope de la pila y almacenarla en la pila
0xA7	GOTO <i>distancia</i>	Ramificación incondicional
0x60	IADD	Sacar dos palabras de la pila; meter su suma
0x7E	IAND	Sacar dos palabras de la pila; meter su AND booleano
0x99	IFEQ <i>distancia</i>	Sacar palabra de la pila y saltar si es cero
0x9B	IFLT <i>distancia</i>	Sacar palabra de la pila y saltar si es menor que cero
0x9F	IF_ICMPEQ <i>distancia</i>	Sacar dos palabras de la pila; saltar si son iguales
0xB4	IINC <i>númvar const</i>	Sumar una constante a una variable local
0x15	ILOAD <i>númvar</i>	Almacenar variable local en la pila
0xB6	INVOKEVIRTUAL <i>despl</i>	Invocar un método
0xB0	IOR	Sacar dos palabras de la pila; almacenar su OR booleano
0xAC	IRETURN	Regresar de método con valor entero
0x36	ISTORE <i>númvar</i>	Sacar palabra de la pila y guardar en variable local
0x64	ISUB	Sacar dos palabras de la pila; almacenar su diferencia
0x13	LDC_W <i>índice</i>	Meter en la pila constante de la reserva de constantes
0x00	NOP	No hacer nada
0x57	POP	Quitar palabra del tope de la pila
0x5F	SWAP	Intercambiar las dos palabras en el tope de la pila
0xC4	WIDE	Prefijo de instrucción; la siguiente instrucción tiene un índice de 16 bits

Figura 4-11. El conjunto de instrucciones de JVM. Los operandos *byte*, *const* y *númvar* son de 1 byte. Los operandos *despl*, *índice* y *distancia* son de 2 bytes.

El diseño del nivel de microprogramación

Microprogramación vertical

Microprogramación horizontal frente a microprogramación vertical

Probablemente la decisión más importante sea cuan codificadas deban estar las microinstrucciones. Se puede hacer funcionar cualquier maquina con n señales de control aplicadas en los lugares adecuados sin decodificar nada.

Este punto de vista nos obliga a considerar un formato de microinstrucción diferente: hacerlo de n bits, uno por señal de control. Las microinstrucciones diseñadas según este principio se denominan **horizontales**. En el otro extremo están las microinstrucciones con un pequeño número de campos muy codificados. Se dice que son **verticales**. Los diseños horizontales tienen un número bastante pequeño de microinstrucciones anchas; los verticales tienen muchas microinstrucciones estrechas.

Hay muchos diseños intermedios. Por ejemplo MAR, MBR, RD, WR y AMUX son bits que controlan directamente funciones del hardware. Por otro lado los campos A,B,C y ALU requieren alguna lógica de decodificación antes de que puedan aplicarse a compuertas individuales. Una instrucción vertical extrema tendría solamente un código de operación, que seria simplemente una generalización de nuestro campo ALU, y algunos operandos, como nuestros campos A,B y C.

Rediseño de microarquitectura vertical(MIC-2)

Cada microinstrucción contendrá ahora tres campos de 4 bits, que suman en total 12 bits. El primer campo es el código de operación, OP, que dice que hace la microinstrucción. Los siguientes campos son dos registros, R1 y R2. Para los saltos, se combinan formando un único campo de 8 bits, R.

Binario Nemotécnico	Instrucción	Significado
0000	ADD	$r1 := r1 + r2$
0001	AND	$r1 := r1 \wedge r2$
0010	MOVE	Mueve registro $r1 := r2$
0011	COMPL	Complementa $r1 := \text{inv}(r2)$
0100	LSHIFT	Desplaza a la izquierda $r1 := \text{desizq}(r2)$
0101	RSHIFT	Desplaza a la derecha $r1 := \text{desder}(r2)$
0110	GETMBR	Almacena el MBR en registro $r1 := \text{rim} \quad \text{--- MBR}$
0111	TEST	Examina registro $\text{if } r2 < 0 \text{ then } n := \text{true}; \text{ if } r2 = 0 \text{ then } z := \text{true}$
1000	BEGRD	Comienza lectura $\text{rdm} := r1; \text{lec} \quad \text{MAR} = \text{rdm}$
1001	BEGWR	Comienza escritura $\text{rdm} := r1; \text{rim} := r2; \text{esc}$
1010	CONRD	Continúa lectura lec
1011	CONWR	Continúa escritura esc
1100		(no usado)
1101	NJUMP	Salta si N = 1 $\text{if } n \text{ then goto } r$
1110	ZJUMP	Salta si Z = 1 $\text{if } z \text{ then goto } r$
1111	UJUMP	Salta siempre $\text{goto } r$

$r = 16 * r1 + r2$

Fig. 4-17. Códigos de operación del Mic-2.

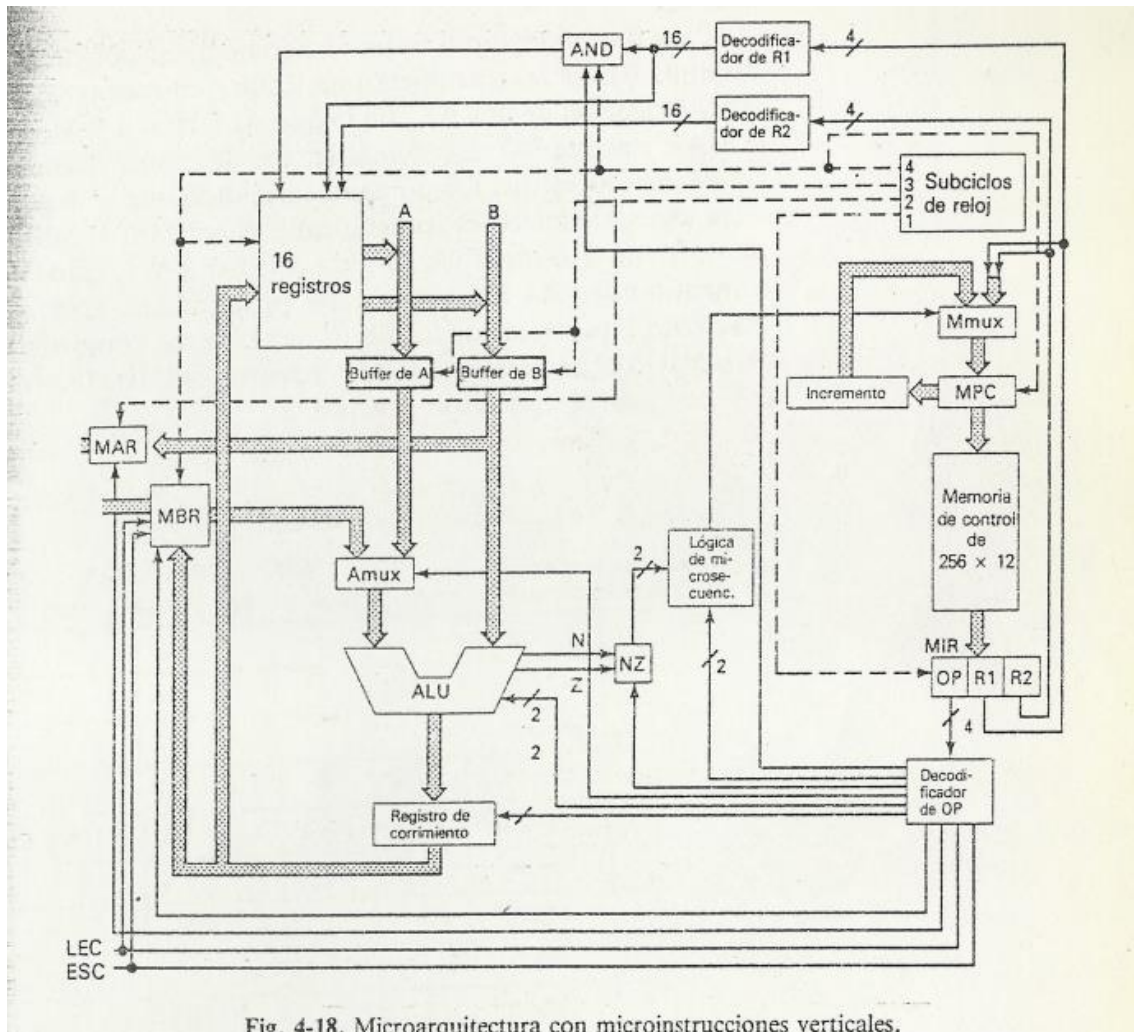


Fig. 4-18. Microarquitectura con microinstrucciones verticales.

La ruta de datos es idéntica a la horizontal. La mayor parte de la porción de control también quedara igual. En particular, aun necesitamos el MIR y la memoria de control (aunque de anchura de 12 bits en lugar de 32). Los tamaños y funciones del MPC, el Mmux, el incrementador, el reloj y la lógica de microsecuenciamiento son idénticos a los del diseño horizontal. Además necesitaremos decodificadores de 4 a 16 para los campos R1 y R2, análogos a los de A,B y C.

Las **tres principales diferencias entre estas dos arquitecturas** son los bloques rotulados AND, NZ y "Decodificador de OP". Se necesita AND porque el campo R1 lleva tanto el bus A como el C. Se presenta el problema de que el bus A se carga durante el subciclo 2 pero el bus C no puede cargarse en la memoria interna hasta que se hayan estabilizado los registros de A y B, en el subciclo 3.

La caja AND hace el Y lógico de cada una de las 16 señales decodificadas con la señal del reloj del subciclo 4 y con la señal que proviene de la decodificación del OP y que equivale a la vieja señal ENC.

El bloque NZ es un registro de dos bits al que se le pueden almacenar las señales N y Z del ALU. Se necesita esta facilidad ya que en el nuevo diseño, el ALU trabaja en una microinstrucción

pero solo puede verificar los bits de estado en la siguiente. Ambas señales se perderían si no se guardaran en alguna parte.

El elemento clave de la nueva microarquitectura es el **decodificador de OP**.

Esta caja toma el campo de código de operación y produce señales para controlar la caja AND, la lógica de microseguenciamiento, NZ, Amux, la ALU, el registro de corrimiento, el MBR, el MAR, RD y WR. La lógica de microseguenciamiento, la ALU y el registro de corrimiento requieren dos señales cada uno, que son idénticas a las del diseño anterior. En total, el decodificador de OP genera 13 señales distintas basándose en los 4 bits más significativos de la microinstrucción en curso.

Para cada uno de los 16 posibles códigos de operación de microinstrucción, el diseñador de la maquina debe determinar cuál de las 13 señales que salen del decodificador de OP está activa y cuál no. Se debe generar una matriz binaria de 16x13 que dé el valor de cada línea de control para cada código de operación.

Código de operación de microinstrucción		Líneas de control											
		ALUL	SHL	AMUX	MAR	RD	MSLH	ALUH	SHH	NZ	AND	MBR	WR
0	ADD				+	+							
1	AND		+		+	+							
2	MOVE	+			+	+							
3	COMPL	+	+		+	+							
4	LSHIFT	+		+	+	+							
5	RSHIFT	+			+	+							
6	GETMBR	+			+	+	+						
7	TEST	+			+								
8	BEGRD	+											
9	BEGWR	+					+	+		+			
10	CONRD	+									+		
11	CONWR	+										+	
12													
13	NJUMP	+											+
14	ZJUMP	+										+	
15	UJUMP	+										+	+

Fig. 4-19. Señales de control para cada código de operación de microinstrucción. Un signo + significa que la señal está activa; un blanco significa que no lo está.

Fig. 4-19. Señales de control para cada código de operación de microinstrucción. Un signo + significa que la señal está activa; un blanco significa que no lo está.

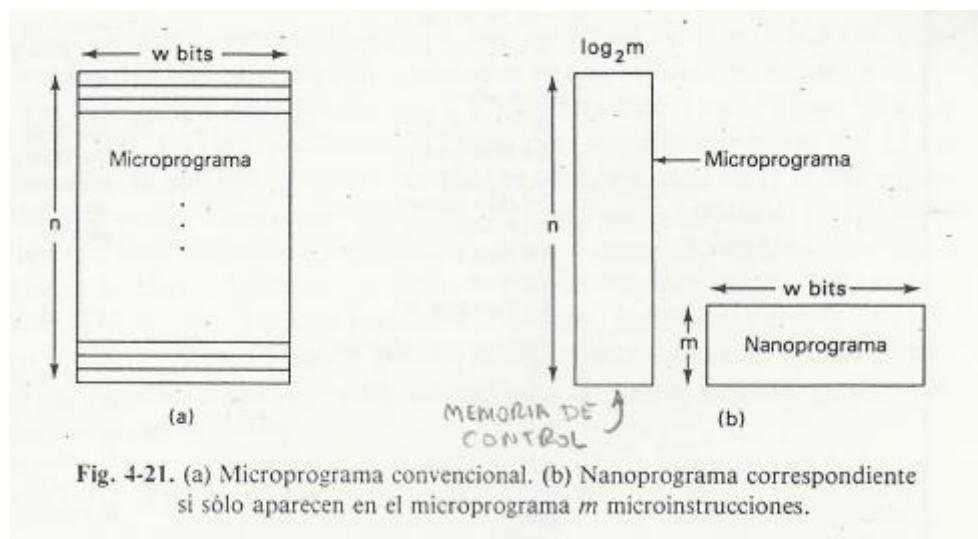
Microprograma

Aunque más largo, este microprograma es más simple que el anterior puesto que cada línea solo realiza una operación. En consecuencia, muchas líneas del horizontal han tenido que

dividirse en dos, tres e incluso cuatro líneas. Otra causa es la ausencia de microinstrucciones de tres direcciones. Utiliza 160 palabras de 12 bits que hacen un total de 1920 bits. La diferencia representa un ahorro de 24% de memoria de control. En una computadora integrada a una pastilla, también representa un ahorro de 24% en el espacio de la pastilla (chip) ocupado por la memoria de control, lo que lo hace más barato y fácil de fabricar. Normalmente, esto hace más lenta la máquina. En consecuencia, las máquinas veloces y caras tienden a ser horizontales y las más lentas y baratas, verticales.

Nanoprogramación

Los diseños vistos hasta ahora tenían 2 memorias: la central y la de control. Con una tercera memoria, la **nanomemoria**, se pueden realizar interesantes compromisos entre la organización horizontal y la vertical. La nanoprogramación es apropiada cuando se repiten varias veces muchas microinstrucciones.



La parte (a) muestra un microprograma de n microinstrucciones de w bits cada una. Se necesitan $n \cdot w$ bits de memoria de control en total para guardarlo. Supongamos que un estudio detallado del microprograma muestra que solo se usan m microinstrucciones diferentes de las 2^w posibles, siendo $m \ll n$. Se podría utilizar una micronanomemoria especial de m palabras de w bits para guardar cada microinstrucción única. En el microprograma original se podría reemplazar cada instrucción por su dirección en la nanomemoria. Como la nanomemoria solo contiene m palabras, la memoria de control necesaria tiene solamente $\log_2 m$ bits de anchura (b).

Ejecucion: Primero se extrae una palabra de la memoria de control y se usa para seleccionar una palabra de la nanomemoria, que se extrae y se guarda en el registro de microinstrucción. Los bits de este registro se utilizan como señales de control durante un ciclo. Al finalizar este ciclo, se extrae la siguiente palabra de la memoria de control y el proceso se repite.

Ejemplo de ahorro: microprograma original 4096×100 solo aparecen 128 microinstrucciones distintas. Bastaría una nanomemoria de 128×100 y la memoria de control tendría ahora 4096×7

Ahorro = $4096 \times 100 - 4096 \times 7 - 128 \times 100 = 368128$ bits

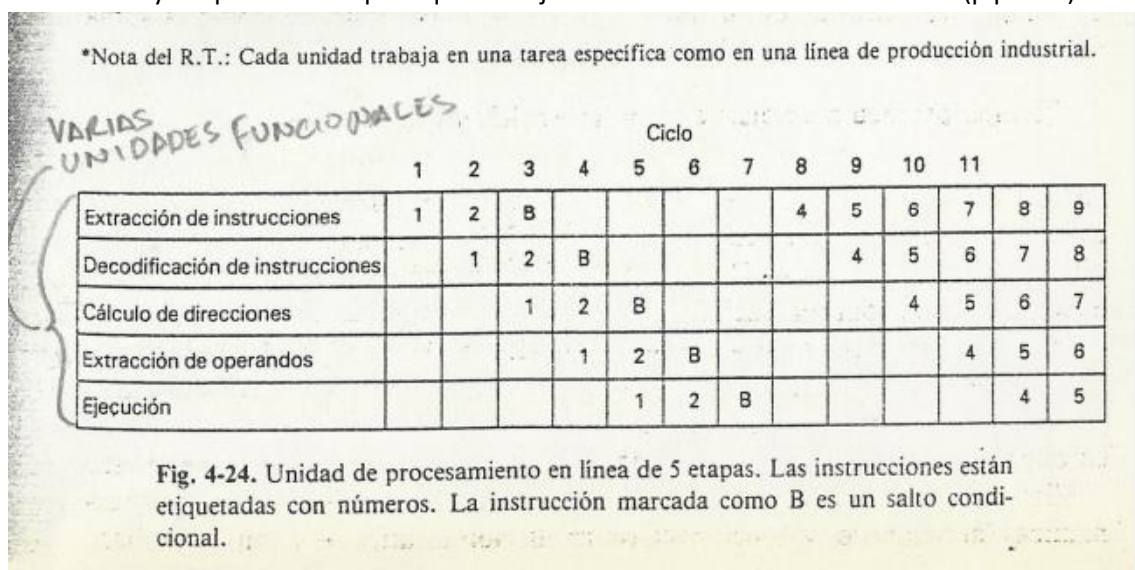
El precio que debe pagarse por el ahorro de memoria es una ejecución mas lenta. La maquina con la memoria de control de dos niveles funcionará con menor rapidez que la original, debido a que el ciclo de extracción requiere ahora dos referencias a memoria, una a la memoria de control y otra a la nanomemoria. Estas dos extracciones no pueden traspapelarse.

La Nanoprogramación es verdaderamente eficaz cuando se usan mucho las mismas microinstrucciones.

Mejora de rendimiento

Procesamiento en línea (pipeline)

Una forma para acelerar la maquina consiste en conformar el hardware con varias unidades funcionales y después unir las para que trabajen en línea o en forma escalonada (pipeline).



Bajo condiciones óptimas, en cada ciclo, después del cuarto ciclo, se ejecuta una instrucción, dando un promedio de ejecución de una instrucción por ciclo, en lugar de una cada cinco ciclos.

Hay estudios que han demostrado que alrededor del 30% de las instrucciones son saltos y estos hacen estragos en la línea de procesamiento. Los saltos pueden calificarse en tres categorías: incondicionales, condicionales e iterativos. Un salto incondicional indica a la computadora que detenga la extracción de instrucciones en forma consecutiva y se dirija a una dirección específica. Un salto condicional verifica una condición y salta si esta se cumple. Las instrucciones iterativas son un caso especial importante de los saltos condicionales, ya que se sabe de antemano, que casi siempre habrá ese salto. (suele ser un contador que cuando llega a cero salta).

Cuando en la línea de procesamiento (pipeline) se encuentra con una instrucción de salto condicional (B en la figura). La siguiente instrucción a ejecutar debería ser la siguiente a la del salto, pero también podría ser la dirección a la cual saltó, llamada **blanco del salto**. En virtud de que el extractor de instrucciones no sabe cual sigue sino hasta que se ejecuta el salto, se demora y no puede continuar hasta la ejecución de este. En consecuencia la línea se vacía.

A la pérdida de cuatro ciclos causada de hecho por el salto se le denomina penalización por salto. Resulta evidente que con un salto en una de cada tres instrucciones, la disminución en el desempeño es sustancial. La máquina corre a menos del 60% de su capacidad potencial, con la probabilidad de 30% de saltos en el código y 65% de saltar en cada salto.

Que puede hacerse para mejorar el desempeño...

Si se pudiera predecir la dirección del salto, se podría extraer la instrucción apropiada (siguiente) y eliminar la penalización.

Existen dos clases de predicciones: estáticas (al tiempo de compilación) y dinámicas (al tiempo de ejecución). Con predicción estática, el compilador hace una suposición para cada una de las instrucciones de salto que genera.

Otro esquema más elaborado para los diseñadores de máquinas consiste en proporcionar dos códigos de operación para cada tipo de salto y hacer que el compilador use el primero si cree que el salto se llevara a cabo y el segundo en caso contrario. (Estático)

En el método de predicción dinámica, el microprograma construye, durante la ejecución, una tabla con las direcciones que contienen saltos y guarda un registro del comportamiento de cada uno. Este método tiende a hacer más lenta la máquina al llevar el registro, de modo que requiere alguna ayuda de hardware.

INTEL 8088

Trayectoria de datos:

El 8088 utiliza una mezcla de micro código y lógica aleatoria para proporcionar un buen funcionamiento, mientras minimiza la extensión de micro código. Las microinstrucciones están en formato **vertical**.

La trayectoria de datos tiene dos secciones, la parte inferior ubicada en dicha mitad de la figura y la parte superior en la mitad más alta

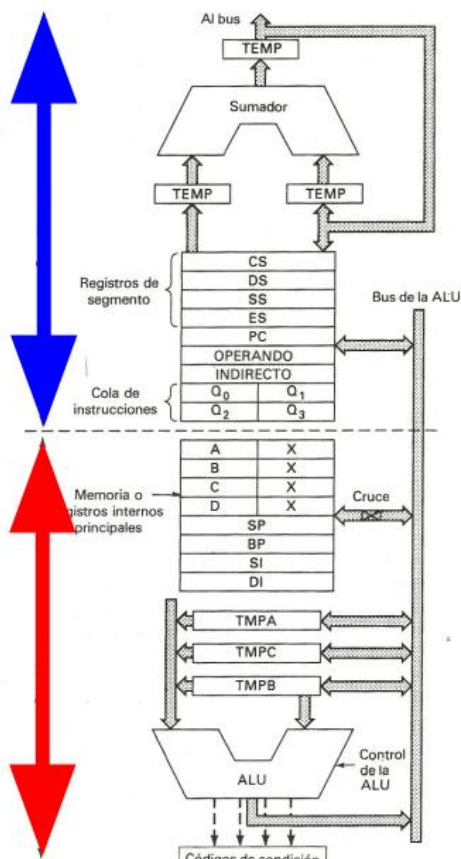
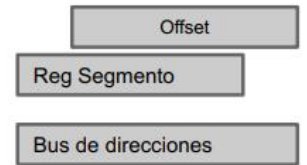


Fig. 4-31. Diagrama altamente simplificado de la trayectoria de datos del 8088.

- La trayectoria de datos se divide en 2
 - Superior p/cálculo de direcciones**
 - RegSegmento + Offset (20 bits)



- Inferior similar a Tanenbaum**
 - ALU multifunción
 - 3 buffer de entrada (TEMPx)
 - 8 registros de uso general
 - N-Z → reg. PSW del microPGM
 - Cruce para operar en 8 bits

La parte inferior contiene una ALU multifunción, que toma dos entradas y produce una salida. Las entradas provienen de tres registros de 16 bits, TMPA, TMPB y TMPC, los cuales se cargan previamente al ciclo de la ALU.

La ALU puede realizar operaciones tanto en 8 bits como en 16. Los códigos de condición resultantes de las operaciones de la ALU se pueden almacenar en el registro **PSW** bajo el control del microprograma.

La parte inferior contiene los 8 registros internos de palabras para AX, BX, CX, DX, SI, DI, BP y SP.

El recuadro marcado como "cruce" tiene la habilidad de intercambiar bytes mientras los copia en cualquier dirección.

La parte superior de la trayectoria de datos está relacionada con la aritmética de las direcciones contiene su propio tablero de registros para contener cuatro registros de segmento (CS, DS, SS, ES), el contador de programa (PC), y dos registros adicionales, también comprende la cola de pre-extracción de instrucciones. También contiene un sumador que se utiliza para combinar porciones de 16 bits de direcciones (offset) y registros de segmento.

Está conectado de modo que los 4 bits de orden inferior del registro índice vayan directamente al bus de direcciones, sin pasar por el sumador, mientras que los bits del 4 al 15 se suman a los bits del 0 al 11 del registro de segmento para formar los 16 bits de orden superior de la dirección del bus

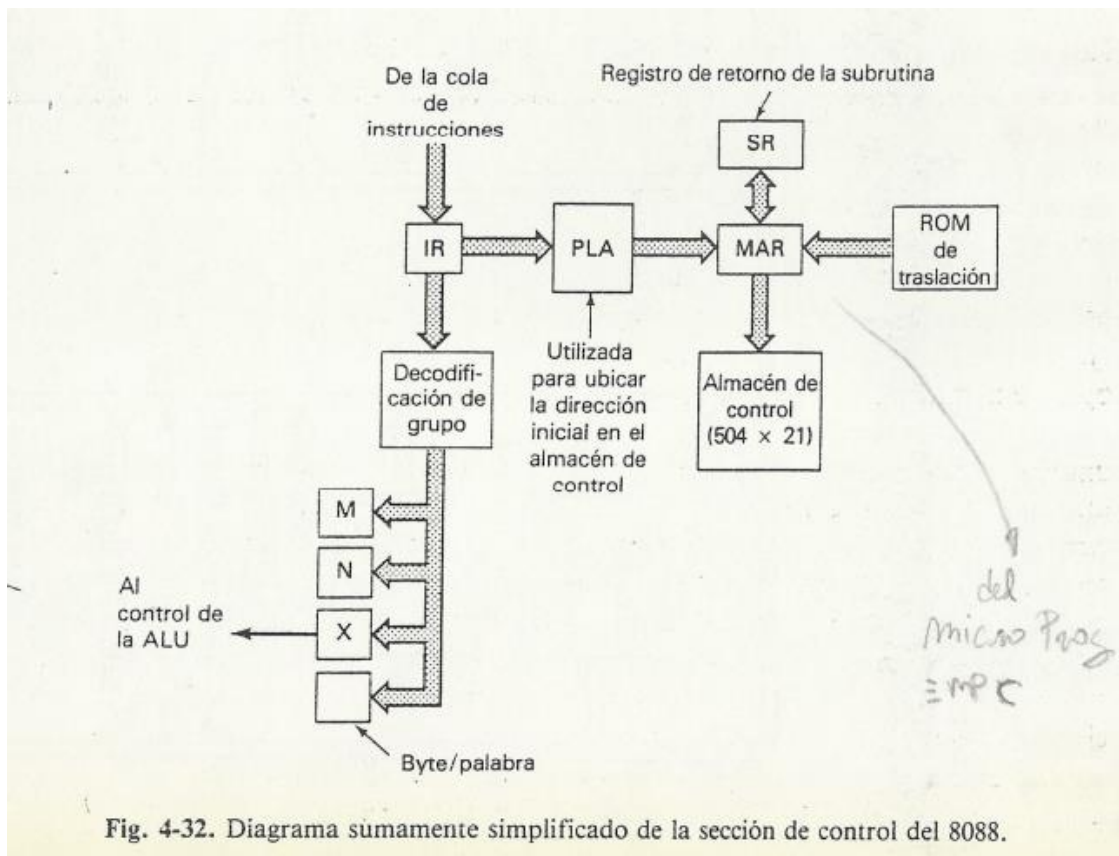
Se proporciona una pequeña ROM para las ctes comunes, por ejemplo, para sumar 1 al PC después de haber extraído un byte de memoria.

El 8088 tiene procesamiento de microinstrucciones en línea (Pipe-Lines). Existen cuatro unidades para pre-extraer instrucciones, decodificarlas, realizar los cálculos de direcciones y ejecutarlas. Las últimas 3 son relativamente convencionales, pero el pre-extractor es en cierta forma poco común. Corre por completo separado del resto del procesador. Siempre que el bus no esté ocupado, el pre extractor envía una solicitud a memoria para leer el siguiente byte en el flujo de instrucciones. Los bytes leídos son almacenados temporalmente en una cola en el tablero de registros superior

Sección de control

La sección de control almacena el microprograma y maneja a la trayectoria. Cuando inicia una nueva instrucción en el nivel de maquina convencional, su byte de código de op se saca de la cola de instrucciones y se carga en el IR

La unidad **decodificador** de grupo extrae información de IR y la disemina por toda la maquina. Los registros M y N toman campos que se utilizan en el cálculo de las direcciones de fuente y destino de los operandos. El registro X contiene la información del código de operación que se utiliza para indicar a la ALU que funciones realizar.



El decodificador de grupo extrae también el bit de byte/palabra del código de operación. Este bit controla si las op de la ALU son en 8 o 16 bits de ancho y si las transferencias hacia o desde el tablero de registros son en 8 o 16 bits.

Las microinstrucciones tienen un ancho de 21 bits. El microprograma en este caso es de 504 x 21. Cuando se extrae una instrucción maquina, el PLA convierte el código de operación en la dirección de inicio de microinstrucción que maneja dicha instrucción.

El microprograma se divide en secuencias de hasta 16 microinstrucciones; cada secuencia maneja una o mas instrucciones de maquina o proporciona un procedimiento de servicio general, como el cálculo de direcciones. Existe dos clases de micro altos: cortos, para saltar adentro de la secuencia actual y largos, para hacerlo a cualquier parte del microprograma. Existen también llamadas a **microprocedimientos** y son similares a los saltos largos, excepto porque depositan su dirección de retorno en el registro **SR**, los microprocedimientos no pueden ser anidados.

La estrategia general para interpretar muchas de las instrucciones aritméticas y lógicas es la siguiente:

Primero se llama a un microprocedimiento para hacer los cálculos de direcciones (Este utiliza los registros M y N). Cuando retorna el microprocedimiento de el cálculo de direcciones, los operandos están disponibles en localidades fijas y el registro X contiene el código de la ALU para esta instrucción maquina. El procedimiento principal ejecuta ahora un ciclo de la trayectoria de datos, con el código de función de la ALU y

los operandos provenientes de los registros. El procedimiento ni siquiera tiene que saber qué operación está realizando.

10		11			
5	5	3	4	3	1
Fuente	Destino	Tipo	ALU	REG	CC

- MicroInstrucción de 21 bits.
- Las 2 partes se pueden ejecutar paralelo.

La parte izquierda contiene 5 campos de 5 bits, permiten a cada microinstrucción realizar un movimiento de registro a registro. Codifica $2^5 = 32$ registros

La parte derecha es una microinstrucción codificada de manera vertical, que consiste en los campos de tipo, código de la ALU, registro y código de condición. Existen seis tipos de microinstrucciones y se distinguen por el campo Tipo:

- Operación de la ALU
- Operación de memoria
- Salto largo
- Salto corto
- Llamada a microprocedimiento
- Contabilidad

Cada microinstrucción se ejecuta en un solo ciclo de reloj, de modo que no es posible mover un valor del conjunto de registros internos a un registro TMP y luego utilizar ese registro como un operando de la ALU en el mismo ciclo.

Los saltos largo y las llamadas a microprocedimientos representan un problema al requerir de más bits de los disponibles en la microinstrucción. La solución es usar una **memoria ROM de traslación** que mapee direcciones de 5 bits en las de todo el microprograma

MOTOROLA 68000

Como se puede apreciar, la “trayectoria de datos” está formada de hecho por tres trayectorias de datos separadas de 16 bits de ancho, cada una de las cuales puede operar en forma independiente a las otras y en paralelo. La parte izquierda maneja los 16 bits de orden superior de la aritmética de direcciones. La parte del medio se hace cargo de los 16 bits inferiores de las direcciones y la parte derecha realiza operaciones en los datos.

En la parte superior de cada uno de los buses se encuentran interruptores, que pueden abrirse o cerrarse por medio de software. Al abrir el interruptor, los buses pueden conectarse eléctricamente.

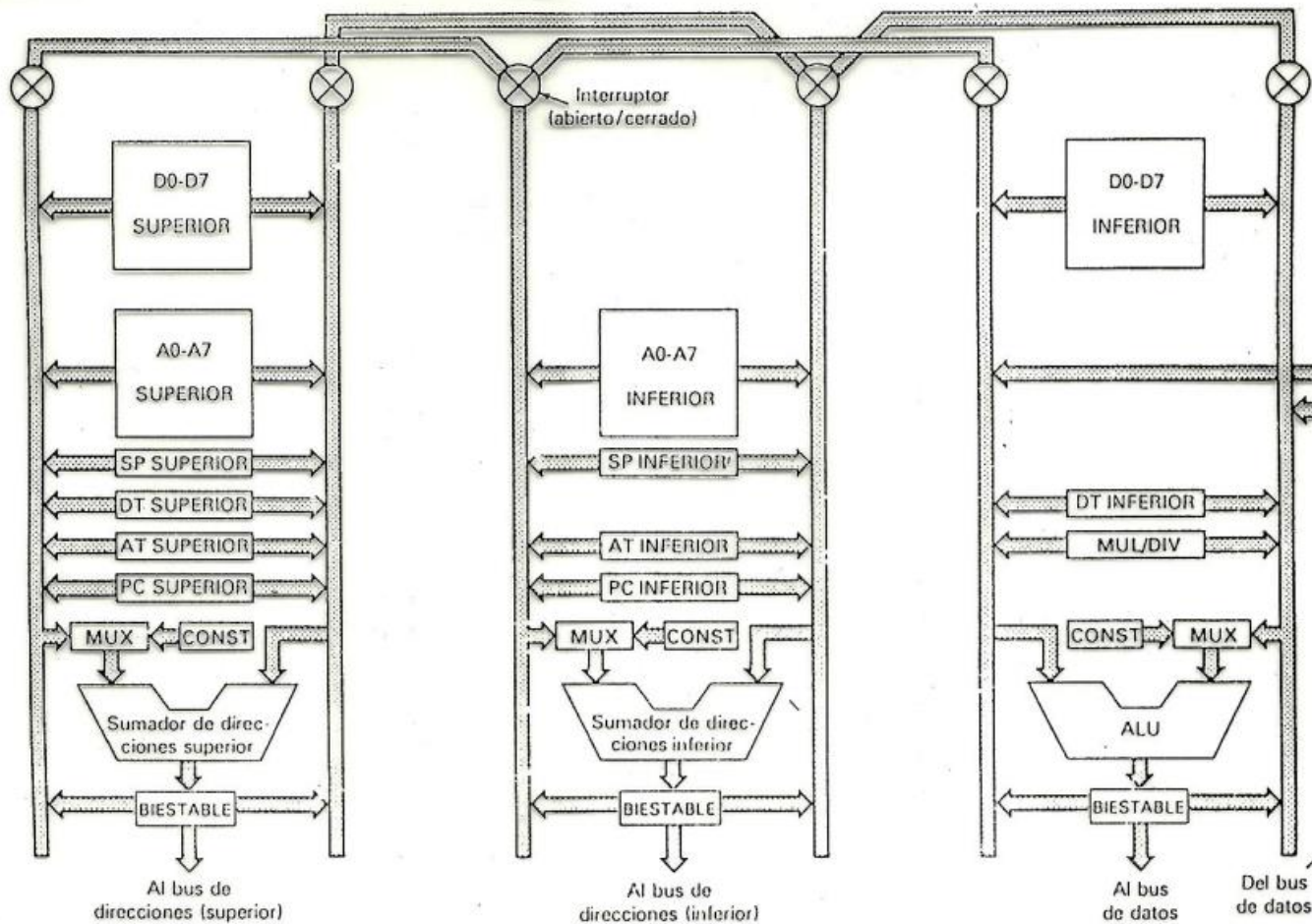


Fig. 4-34. Diagrama sumamente simplificado de las trayectorias de datos del 68000.

El recuadro marcado como **D0-D7 SUPERIOR** contiene los 16 bits de orden superior de los ocho registros de datos. El recuadro bajo el anterior, contiene los 16 bits de orden superior de los ocho registros de direcciones (A0-A7). En seguida esta el SP del modo supervisor, dos registros de utilería: TDAT (Datos temporal) y TDIR (Dirección temporal) y la mitad superior del contador de programa (PC). Ambos buses conducen a un sumador de 16 bits. Nótese que la entrada izq del sumador es un multiplexor (MUX), que puede seleccionar el bus de la izquierda o la parte superior de una ROM de constantes (CONST) bajo el control del microprograma.

La salida de la ALU tiene un biestable y se puede dirigir a cualquiera de los buses, de modo que los resultados se pueden almacenar en los registros. El biestable puede conducir el bus de dirección externa para salir de la parte superior de una dirección de memoria de 32 bits.

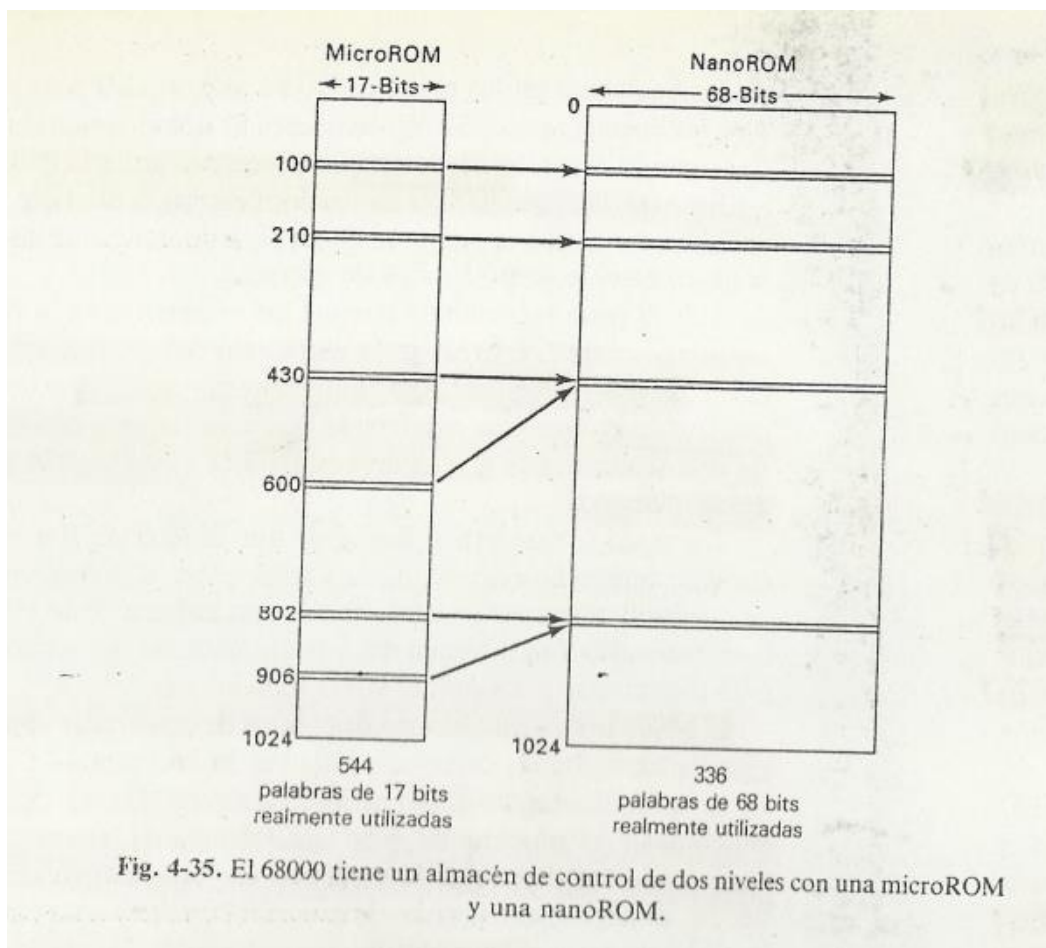
La parte media, no contiene los registros internos de datos o el registro TDAT. La ROM de constantes contiene, los 16 bits de orden inferior de estas y no los de orden

superior. La salida del biestable puede conducir el bus de direcciones de orden inferior. Es posible habilitar ambas partes en forma simultanea.

La parte de la derecha contiene la porción de orden inferior de los registros internos de datos y la misma porción de TDAT. También tiene una ROM de constantes, se encuentra en el otro bus y contiene otras constantes. Existe un registro extra (MUL/DIV) que se requiere para las operaciones de multiplicación y división. **Aunque no se muestre en la figura** existe un conjunto de tres registros para almacenar las instrucciones recién arribadas.

Como el 68000 tiene el procesamiento en línea (pipeline) se necesita de los tres. El primero de los registros contiene la información de la instrucción que se esta ejecutando; el segundo la instrucción que se está decodificando; y el tercero la que se está extrayendo de memoria.

El 68000 utiliza un almacén de control de dos niveles con un microprograma y un nano programa. Los detalles son en cierto modo diferentes que el ejemplo dado en el texto, pero la meta es la misma: reducir el número de bits en el almacén de control haciendo que el microprograma determine la secuencia de nano instrucciones ejecutadas.



El microprograma consiste en 544 palabras de 17 bits y el nano programa de 336 palabras de 68 bits, para un total de 32096 bits. Esta implementación representa un ahorro del 13%.

Las memorias ROM tanto para el microprograma como para el nano programa están direccionadas por números de 10 bits, que permiten entradas en cada uno de ellos. En la nanoprogramacion convencional, el hardware extrae primero una microinstrucción, y esta contiene la dirección de la nanoinstruccion a utilizar. Este es un proceso inherente de dos etapas, ya que no se puede extraer la nanoinstruccion hasta que esté disponible la microinstrucción.

Los diseñadores del 68000 utilizaron un truco para hacer todo en un mismo ciclo. En la mayoría de los casos, la nanoinstruccion correspondiente a la microinstrucción que esta en la dirección n , se encuentra en la misma dirección, de modo que ambos se pueden extraer al mismo tiempo.

En los casos en que dos o más microinstrucciones comparten una misma nanoinstruccion, se ha quitado un transistor del circuito decodificador de nanoinstrucciones para mapear dos o más direcciones en la misma palabra.

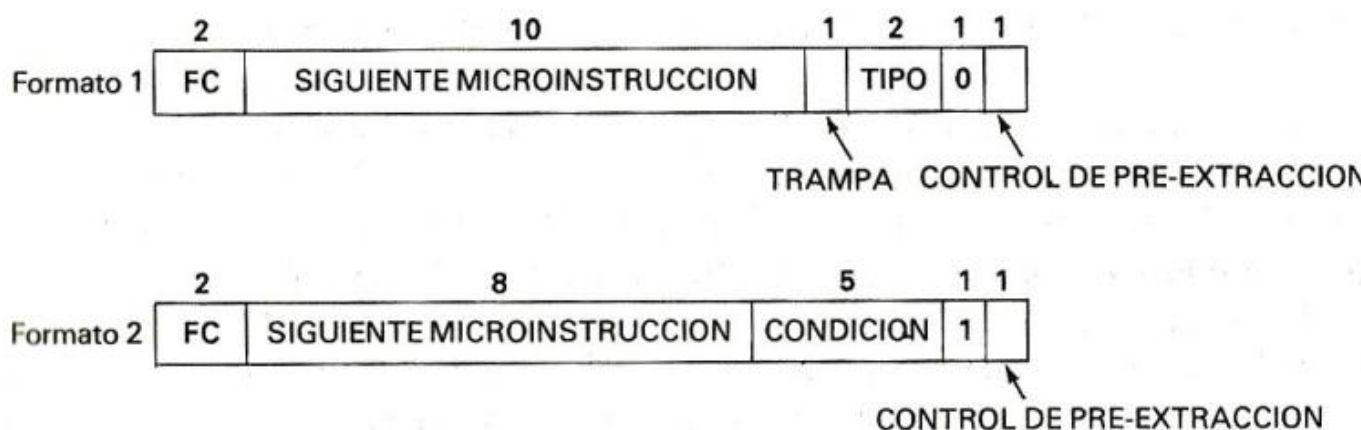


Fig. 4-36. Formatos de instrucciones del 68000.

En la figura se muestran los dos formatos de microinstrucciones, estas no se ejecutan en forma secuencial como en el 8088. En vez de ello, cada una nombra a su sucesora.

El formato 1 se utiliza para todas las microinstrucciones con excepción de saltos condicionales, los cuales usan el formato 2. Ambos tienen un bit que se emplea para iniciar la **pre-extracción** de la siguiente instrucción. Además tienen en el otro extremo 2 bits (FC), para enviar señales a las patas de la pastilla que indican si una referencia a memoria representa el espacio de instrucciones, el espacio de datos, la recepción de interrupciones o ninguna de estas.

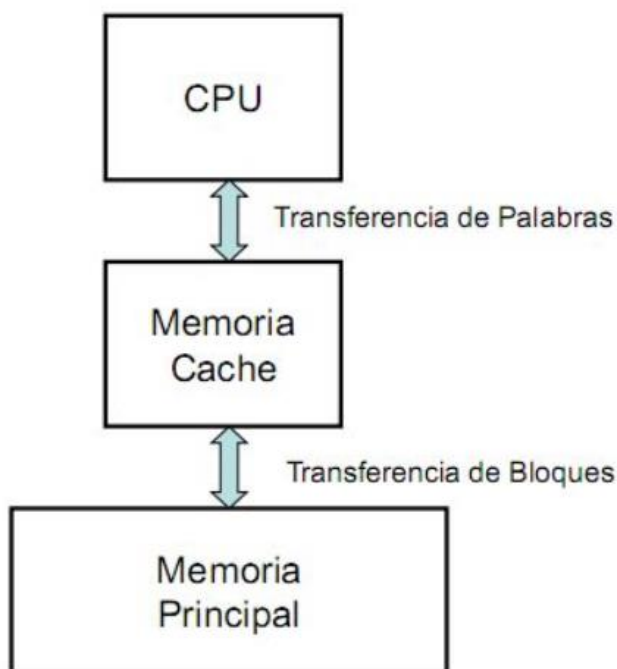
El formato 1 contiene unos cuantos bits de uso diverso y una dirección de 10 bits que indica de donde extraer la siguiente microinstrucción. El formato 2 contiene un campo de 5 bits que selecciona una de las 32 condiciones preestablecidas para probar, tales

como códigos de condición y bits en el registro de instrucciones. El resultado de la condición produce 2 bits que se combinan con los 8 bits de la microinstrucción para selecciones uno de los cuatro sucesores potenciales.

Las nanoinstrucciones de 68 bits están codificadas en forma horizontal con 38 campos distintos, la mayoría de ellos de uno o más bits. Los campos controlan las compuertas de todos los registros y las transferencias de los registros internos a los buses, la selección de registros, el control de interruptores, el establecimiento de códigos de condición, la función de la ALU, el uso de memorias ROM de constantes y multiplexores, así como la operación de la memoria

Memoria cache

A la memoria pequeña y rápida se le llama **cache** y está bajo el control del microprograma



Es sabido que los programadores no acceden a las memorias en forma completamente aleatoria. Si una referencia a memoria dada, es a la dirección A, es común que la siguiente referencia se realice en los alrededores de A.

Se llama **principio de localidad** a la observación de que las referencias a memoria realizadas en un intervalo de tiempo corto, tienden a usar solo una fracción de la memoria total y representan la base para todos los sistemas de memoria cache. La idea general es que cuando una palabra es referenciada, se le trae de la memoria (RAM o quizás un disco) hacia la cache, de modo que la siguiente que se utilice, se pueda acceder muy rápido.

Si una palabra se lee o se escribe k veces en un intervalo corto, la computadora necesitara de 1 referencia a la memoria lenta y $k-1$ referencias a la memoria rápida. Entre más grande es k , mejor es el desempeño general. Podemos formalizar este cálculo llamando c al tiempo de acceso a la memoria cache, m al tiempo de acceso a la memoria principal y h a la **proporción de aciertos**, que es la fracción de todas las referencias que pueden ser satisfechas fuera de la cache.

$$h = (k-1) / k$$

Algunos autores definen también la **proporción de fallas** como $1-h$. Con estas definiciones podemos calcular el tiempo medio de acceso de la siguiente manera:

$$\text{Tiempo medio de acceso} = c + (1-h)m$$

en este caso m es el tiempo de acceso a la memoria lenta

Se utilizan dos formas diferentes de organización de la cache, junto con una tercera que es un híbrido de las dos primeras. Para los tres tipos se asume que la memoria sea de 2^m bytes, dividida (de manera conceptual), en bloques consecutivos de b bytes, lo que da un total de $2^m/b$ bloques. Cada bloque tiene una dirección que es múltiplo de b . El tamaño b del bloque es por lo general una potencia de dos.

El primer tipo es la **memoria cache asociativa**, consiste en un cierto número de **renglones** o **líneas**, cada uno de los cuales contiene un bloque y su número correspondiente, junto con un bit de *validez* que indica si el renglón está actualmente en uso o no.

Dir	Bloque	Bit de Validez	Nro de Bloque	Valor
0	137 0	1	0	137
4	52 1	1	600	2131
8	1410 2	1	2	1410
12	635 3	0		
16		1	160248	290380
...	...	...	...	...
2^{24}	2^{22}	1	22	32

1024 líneas

Se ilustra una cache de 1024 líneas y una memoria con 2^{24} bytes, divididos en 2^{22} bloques de 4 bytes.

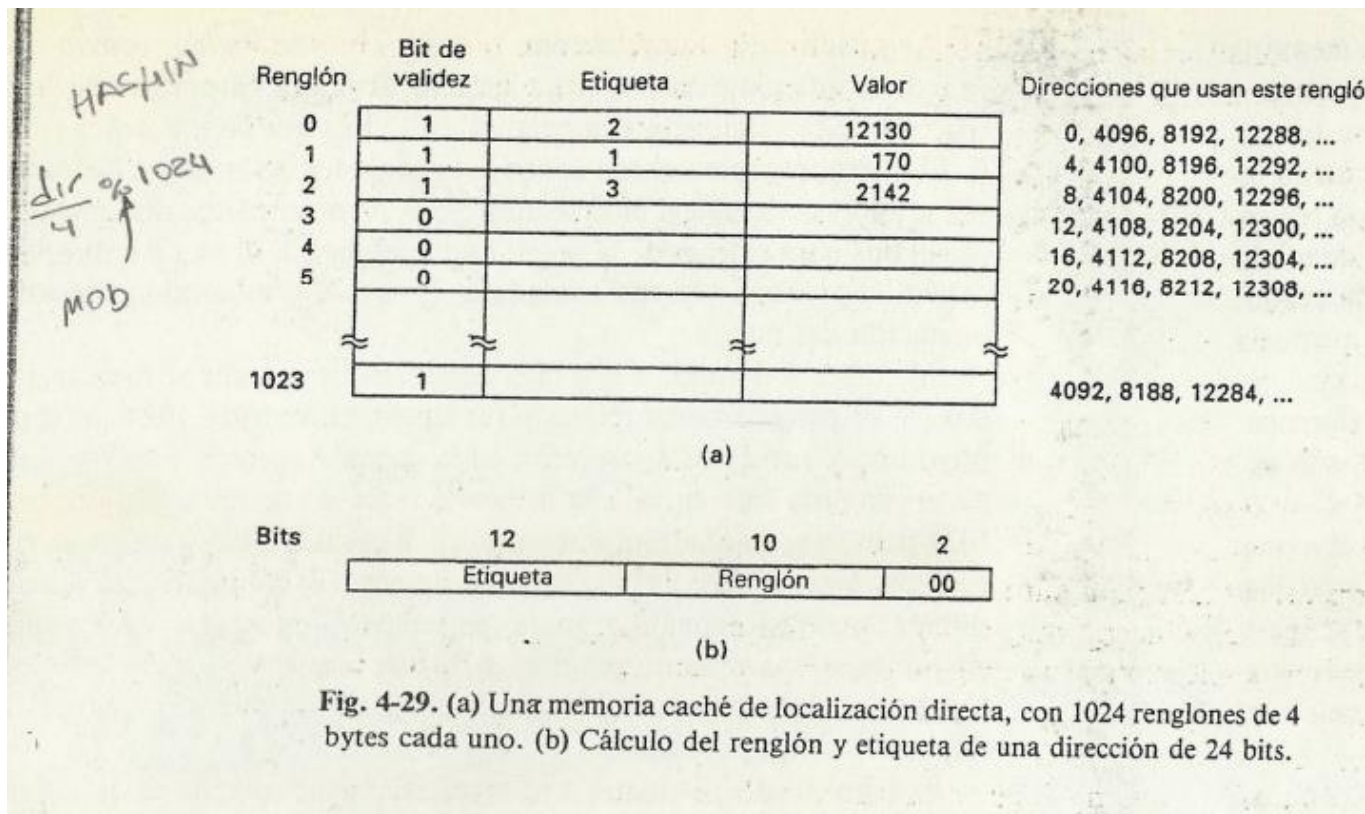
En este tipo de cache el orden de los accesos es aleatorio. Si se necesita mas de 1024 palabras, en algún momento la cache estará llena y un renglón previo deberá descartarse para dar lugar a uno nuevo.

Este renglón posiblemente se seleccione de manera aleatoria.

El aspecto que distingue a la memoria cache asociativa de los otros tipos, consiste en que cada renglón contiene el número de bloque y su entrada.

Para evitar reducir el costo, se invento la **memoria cache de mapeo directo**. Este tipo evita la búsqueda poniendo cada bloque en un renglón cuyo número se puede calcular fácilmente a partir del número del bloque. Por ejemplo, el numero de renglón puede ser el numero de bloque *modulo* del numero de renglones.

Con bloques de 4 bytes (una palabra) y 1024 renglones, el número de renglón que corresponde a la dirección A es igual a $(A/4) \text{ modulo } 1024$ (hash). Mientras que las memorias cache de mapeo directo eliminan el problema de buscar; crean por otro lado un nuevo problema; como indicar cuál de las muchas palabras que corresponden al renglón lo está ocupando. En efecto se han creado 1024 clases de equivalencia basada en los números de bloque *modulo* del tamaño de la cache. En el ejemplo, el renglón 0 puede contener a cualquiera de las palabras 0, 4096, 8192, etc.



La forma de señalar cual está en un momento dado en dicho renglón es poner parte de la dirección en el campo *etiqueta* de la cache. Este campo contiene aquella parte de la dirección que no puede calcularse a partir del número de renglón.

A la cache con varios registros por renglón se la denomina **memoria cache asociativa agrupada**.

Rgn	B Etq Valor			B Etq Valor						B Etq Valor		
0												
1												
2												
3												
4												
	1	12	32	1	12	32	1	12	32	1	12	32

Esta reduciría el número de colisiones que tendría la de mapeo directo.

La asociativa y mapeo directo son casos especiales de la cache asociativa agrupada.

Hasta ahora hemos analizado cómo se comporta la memoria caché en la operación de lectura, pero también debemos analizar qué pasa cuando la CPU escribe en la memoria.

Existen dos grandes estrategias:

- Escritura a Memoria (write through) la operación de escritura se hace en la memoria caché y también en la memoria principal. (Usa mucho BUS)
- Retrocopiado (write back) la operación de escritura se hace solo en la memoria caché y cuando se elimina o reemplaza el renglón en la caché se copia en la memoria principal. (Buffer de archivos desactualizados)

Memoria virtual

A mitad del siglo XX los programadores dedicaban gran parte de su tiempo a lograr que sus programas entraran en una diminuta memoria. En muchos casos era necesario usar un algoritmo que trabajaba mucho más lento que otro mejor, simplemente porque el mejor era demasiado grande. La solución tradicional a este problema era usar una memoria secundaria, como un disco. El programador dividía el programa en varios fragmentos, llamados **superposiciones**. Aunque se usó durante muchos años,

esta técnica implicaba mucho trabajo invertido en la gestión de superposiciones. En 1961 un grupo de investigadores de Manchester, propuso un método para realizar automáticamente el proceso de superpone, siendo, ahora conocido como **memoria virtual** que tenía la ventaja obvia de liberar al programador de una gran cantidad de molestas tareas de contabilidad.

Paginación

La idea de este grupo fue la de separar los conceptos de espacios de direcciones y posiciones de memoria.

Consideremos el siguiente ejemplo, una computadora representativa de esa época, con un campo de dirección de 16 bits en sus instrucciones y 4096 palabras de memoria. Un programa de esta computadora podía direccionar 65536 palabras de memoria (2^{16}). Cabe señalar que el número de palabras direccionales depende únicamente del número de bits que tiene una dirección y nada tiene que ver con el número de palabras de memoria que realmente cuenta. El espacio de direcciones de esta computadora consiste en los números 0, 1, 2, ..., 65535, porque ese es el conjunto de direcciones posibles. Sin embargo, la computadora bien podría tener menos de 65535 de palabras de memoria.

La idea de separar el espacio de direcciones y las direcciones de memoria es la siguiente.

En cualquier momento dado, es posible acceder directamente a 4096 palabras de memoria, pero no es necesario que correspondan a las direcciones de memoria 0 a 4095. Podríamos, por ejemplo, “decirle” a la computadora que, en adelante, cada vez que se haga referencia a la dirección 4096, se usara la palabra que está en la dirección 0. Cada vez que se haga referencia a la dirección 4097, se usara la palabra de memoria que está en la dirección 1.

En otras palabras, habremos definido una correspondencia o “mapeo” del espacio de direcciones a las direcciones de memoria reales, como se muestra:

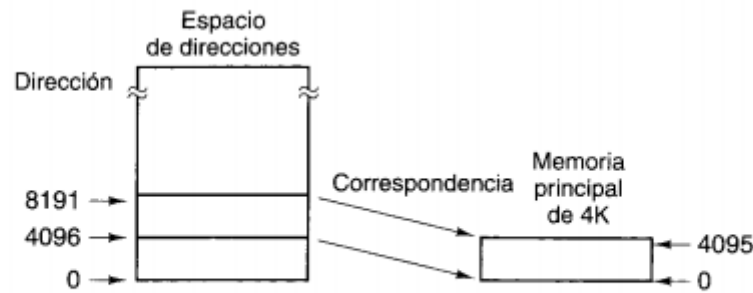


Figura 6-2. Mapeo en el que las direcciones virtuales 4096 a 8191 se hacen corresponder con las direcciones 0 a 4095 de la memoria principal.

Una maquina de 4K sin memoria virtual simplemente tiene una correspondencia fija entre las direcciones 0 a 4095 y las 4096 palabras de memoria. Una pregunta interesante es: “¿Que sucede si un programa ramifica a una dirección entre 8192 y 12287?” en una maquina sin memoria virtual, el programa imprimiría un mensaje como “Referencia a memoria inexistente” y terminaría el programa. En una maquina con **memoria virtual**. En una maquina con **memoria virtual**, ocurriría lo siguiente:

1. El contenido e la memoria principal se guardaría en el disco.
2. Se localizarían en el disco las palabras de 8197 a 12287.
3. Se cargarían en la memoria principal las palabras 8192 a 12287
4. El mapa de direcciones se modificaría para hacer corresponder las direcciones 8192 a 12287 con las posiciones de memoria 0 a 4095.
5. La ejecución continuaría como si nada fuera de lo normal hubiera ocurrido.

Esta técnica de superposición automática se denomina **paginación**, y los fragmentos de programas que se leen del disco se llaman **páginas**.

Llamaremos a las direcciones a las que el programa puede hacer referencia **espacio de direcciones virtual** y a las direcciones de memoria reales, en hardware, **espacio de direcciones físico**.

Una tabla de páginas relaciona las direcciones virtuales con las direcciones físicas. Suponemos que hay suficiente espacio en el disco para guardar todo el espacio de direcciones virtuales. Los programas se escriben como si hubiera suficiente memoria principal para todo el espacio de direcciones virtual, aunque no sea así. Los programas pueden cargar valores de, o guardarlos en, cualquier palabra del espacio de direcciones virtual, o saltar a cualquier dirección situada en cualquier punto del espacio de direcciones virtual, sin tener en cuenta el hecho de que en realidad no hay suficiente memoria física. De hecho, el programador puede escribir sin siquiera saber **si existe o no** memoria virtual.

Implementación de la paginación

El espacio de direcciones se divide en varias páginas de tamaño uniforme. El tamaño de la página siempre es una potencia de 2. El espacio de direcciones físico se divide en fragmentos de la misma manera, y cada uno es del mismo tamaño de una página, de modo que cada fragmento de la memoria principal puede contener exactamente una página. Estos fragmentos se denominan **marcos de página (frames)**

Página	Direcciones virtuales		
15	61440 – 65535		
14	57344 – 61439		
13	53248 – 57343		
12	49152 – 53247		
11	45056 – 49151		
10	40960 – 45055		
9	36864 – 40959		
8	32768 – 36863		
7	28672 – 32767		
6	24576 – 28671		
5	20480 – 24575		
4	16384 – 20479		
3	12288 – 16383		
2	8192 – 12287		
1	4096 – 8191		
0	0 – 4095		

(a)

Marco de página	32K bajos de la memoria principal Direcciones físicas
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

(b)

Figura 6-3. (a) Los primeros 64K del espacio de direcciones virtual divididos en 16 páginas, cada una de las cuales tiene 4K. (b) Memoria principal de 32K dividida en ocho marcos de página de 4K cada uno.

Toda computadora con memoria virtual tiene un dispositivo para efectuar el mapeo virtual a físico, llamado **unidad de gestión de memoria (MMU)**

Para ver cómo funciona la MMU considere el siguiente ejemplo:

Cuando la MMU recibe una dirección virtual de 32 bits, la divide en un número de página virtual, de 20 bits y una distancia dentro de la página de 12 bits (Porque en nuestro ejemplo las páginas son de 4K), el número de página virtual se usa como índice de la tabla de páginas para encontrar la entrada que corresponde a la página que se hizo referencia. Lo primero que hace la MMU con la entrada de la tabla de páginas es verificar si la página a la que hizo referencia está actualmente en la memoria principal. La MMU efectúa esta verificación examinando el bit **presente/ausente** de la entrada

de la tabla de páginas, en nuestro ejemplo el bit es 1, lo que implica que la pagina actualmente está en la memoria. El siguiente paso es tomar el valor de marco de página en la entrada seleccionada (En este caso 6) y copiarlo en los tris bits superiores del registro de salida de 15 bits, los 12 bits de orden bajo de la dirección virtual se copian en los 12 bits de orden bajo del registro de salida, como se muestra. Esta dirección de 15 bits se envía entonces a cache o a la memoria para buscarla.

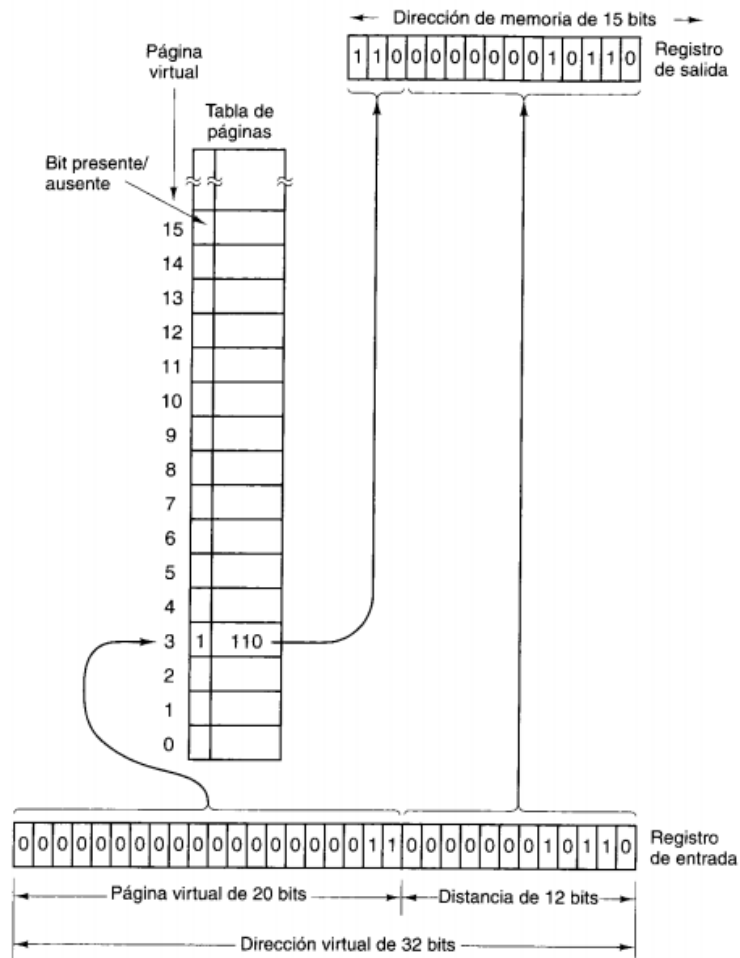


Figura 6-4. Formación de una dirección de memoria principal a partir de una dirección virtual.

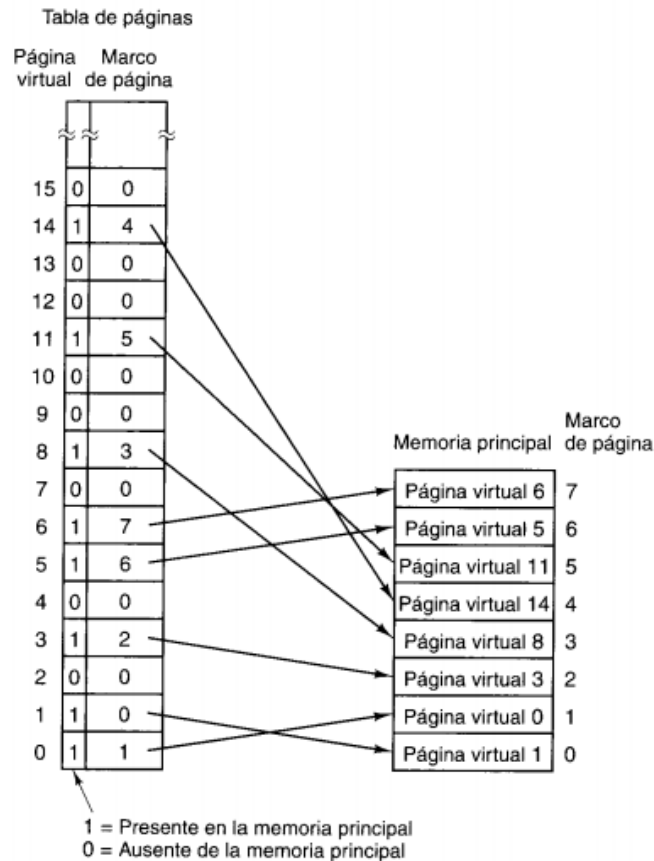


Figura 6-5. Una posible correspondencia entre las primeras 16 páginas virtuales y una memoria principal con ocho marcos de página.

Paginación por demanda

En la explicación anterior se supuso que la página virtual a la que se hizo referencia estaba en la memoria principal. Sin embargo esto no siempre ocurre. Cuando se hace una referencia a una dirección que está en una página que no está presente en la memoria principal, ocurre un **fallo de página**.

La **paginación por demanda** significa que cada página de un proceso (Proceso = programa en ejecución) se introduce en la memoria principal solo cuando este se necesita. La pregunta de si debe usarse o no la paginación por demanda es solo pertinente cuando se inicia un programa. Una vez que el programa ha estado trabajando durante cierto tiempo, las páginas requeridas ya se habrán reunido en la memoria principal. Si la computadora es de tiempo compartido y los procesos se intercambian a disco después de ejecutarse 100 ms (digamos), cada programa se reiniciara muchas veces durante su ejecución. Puesto que el mapa de memoria es único para cada programa, y cambia cuando se conmutan los programas, como en un sistema de tiempo compartido, la pregunta se vuelve cuestión crítica.

La estrategia alternativa se basa en la observación de que la mayor parte de los programas no hace referencia a su espacio de direcciones de manera uniforme, si no

que las referencias tienden a concentrarse en un número reducido de páginas. Una referencia a memoria podría traer una instrucción, podría traer datos o podría guardar datos. En cualquier instante t existe un conjunto formado por todas las páginas utilizadas por las k referencias a memorias recientes. Esto se llama **conjunto de trabajo**.

Política de reemplazo de páginas

Idealmente, el conjunto de páginas que un programa está usando activa e intensivamente llamado **conjunto de trabajo**, se puede mantener en la memoria a fin de reducir los fallos de página. Sin embargo, los programadores casi nunca saben cuales páginas están en el conjunto de trabajo, por lo que el SO debe descubrir dicho conjunto dinámicamente. Cuando un programa hace referencia a una página que no está en la memoria principal, la página requerida debe traerse del disco. Sin embargo, para que entre generalmente es necesario enviar alguna otra página devuelta al disco. Se necesita un algoritmo que decida cual debe quitarse.

No es una buena idea escoger al azar la página que se quitara.

Un algoritmo muy utilizado expulsa la página que se usó menos recientemente porque la probabilidad *a priori* de que no esté en el conjunto de trabajo vigente es alta. Este algoritmo es el **LRU (menor reciente utilizada, Least Recently Used)**. Aunque este algoritmo normalmente funciona bien, existen situaciones patológicas donde este falla.

Imagine un programa que ejecuta un ciclo grande que se extiende sobre nueve páginas virtuales en una máquina que tiene espacio solo para ocho páginas en la memoria física. Una vez que el programa llega a la página 7, el estado de la memoria principal es que se muestra en la figura 6-6(a). En algún momento se intenta traer una instrucción de la página virtual 8, lo que causa un fallo de página. Se debe tomar una decisión sobre cual página se expulsará. El algoritmo LRU escoge la página virtual 0 porque es la que se usó menos recientemente. Se quita la página virtual 0 y se trae la página virtual 8 para sustituirla, y la situación es la que se muestra en la figura 6-6(b)

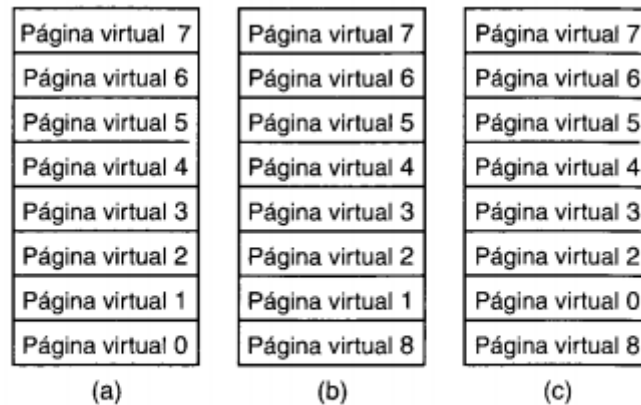


Figura 6-6. Fracaso del algoritmo LRU.

Después de ejecutar las instrucciones de la página virtual 8, el programa salta al principio del ciclo, a la página virtual 0. Este paso causó otro fallo de página. La página virtual 0 acaba de expulsarse, se tiene que volver a traer. El algoritmo LRU escoge la página 1 para expulsión, lo cual lleva a la situación 6-6(c). El programa continuara en la pagina 0 durante un rato y luego tratara de traer la instrucción de la pagina virtual 1, cuando un fallo de pagina. Hay que volver a traer la pagina 1 y para eso se expulsara la página 2.

Ya deberá ser obvio que el algoritmo está tomando la peor decisión en cada ocasión. Sin embargo si la memoria principal disponible es mayor que el tamaño del conjunto de trabajo, el algoritmo de LRU si tiende a minimizar el numero de fallos de pagina.

Otro algoritmo de reemplazo de páginas es el de **primero en entrar, primero en salir (FIFO)**, FIFO expulsa la pagina que se cargo menos recientemente, sin importar cuando fue la última vez que se hizo referencia a ella.

Si el conjunto de trabajo es mayor que el número de marcos de página disponibles, ningún algoritmo que no sea un oráculo dará buenos resultados, y los fallos de página serán frecuentes. Un programa que genera fallos de página de forma continúa y frecuente esta **hiperpaginado**.

Tamaño de página y fragmentación.

Si por casualidad el programa y los datos del usuario llenan exactamente un número entero de páginas, no se desperdiciara espacio cuando estén en la memoria. Pero esto no sucede casi nunca así, quedara siempre cierto espacio sin utilizar en la última página.

El problema de estos bytes desperdiciados se denomina **fragmentación interna**

Segmentación (sacado del libro de stalling, el de tannenbaum esta parte tiene más relleno que naruto)

Hay otra forma en la que puede subdividirse la memoria direccionable, conocida como **segmentación**.

Mientras que la paginación es invisible para el programador y sirve para proporcionar al programador un espacio de direcciones mayor, la segmentación es usualmente visible para él y proporciona una forma conveniente de organizar los programas y los datos, para asociar los privilegios y los atributos de protección con las instrucciones y los datos.

La segmentación permite que el programador vea la memoria constituida por múltiples espacios de direcciones o segmentos. Los segmentos tienen un tamaño variable, dinámico. Usualmente el programador o el SO asignaran programas o datos a segmentos distintos. Puede haber segmentos de programa distintos para varios tipos de programas y también distintos segmentos de datos. Se pueden asignar a cada segmento derecho de acceso y uso. Las referencias a memoria se realizan mediante direcciones constituidas por un número de segmento y un desplazamiento.

Esta organización tiene ciertas ventajas para el programador frente a un espacio de direcciones no segmentado:

1. Simplifica la gestión de estructuras creciente de datos. Si el programador no conoce *a priori* el tamaño que puede llegar a tener una estructura de datos particular, no es necesario que lo presuponga. A la estructura de datos se le asigna su propio segmento y el SO lo expandirá o lo reducirá según sea necesario.
2. Permite modificar los programas y recompilarlos independientemente, sin que sea necesario volver a enlazar y cargar el conjunto entero de programas. De nuevo, esto se consigue utilizando varios segmentos.
3. Permite que varios procesos compartan segmentos. Un programador puede situar un programa correspondiente a una utilidad o una tabla de datos de interés en un segmento que puede ser direccionado por otros procesos.
4. Se facilita la protección. Puesto que un segmento se construye para contener un conjunto de programas o datos bien definido, el programador o el administrador del sistema puede asignar privilegios de acceso de forma adecuada.

Estas ventajas no se tienen con la paginación, que es invisible para el programador. Por otra parte hemos visto que la paginación proporciona una forma eficiente de gestionar la memoria. Para combinar las ventajas de ambas, algunos sistemas están equipados con el hardware y el software del sistema operativo que permite las dos.

Implementación de la segmentación

Hay dos formas de implementar la segmentación: intercambio y paginación. En el primer esquema, cierto conjunto de segmentos están en la memoria en un momento dado. Si se hace referencia a un segmento que no está en memoria, se trae este segmento. Si no hay espacio para él, será preciso escribir primero en el disco uno o

más segmentos. En cierto sentido, el intercambio de segmentos es parecido a la paginación por demanda, los segmentos salen y entran conforme se van necesitando.

Sin embargo, la implementación de la segmentación difiere de la de la paginación en un aspecto **fundamental**: *las páginas tienen un tamaño fijo y los segmentos no*.

La siguiente figura muestra una memoria física que inicialmente contiene 5 segmentos. Considere que sucede si el segmento 1 se expulsa y se coloca en su lugar el segmento 7, que es más pequeño. Llegamos a la configuración de memoria de la figura 6-10(b). Entre el segmento 7 y el 2 hay un área desocupada. Luego el segmento 4 es sustituido por el segmento 5, como en la figura 6-10(c), y el segmento 3 es sustituido por el segmento 6, como en la figura 6-10(d). Después de un rato de funcionar el sistema, la memoria estará muy dividida, con varios segmentos y muchos huecos. Este fenómeno se llama **fragmentación externa**.

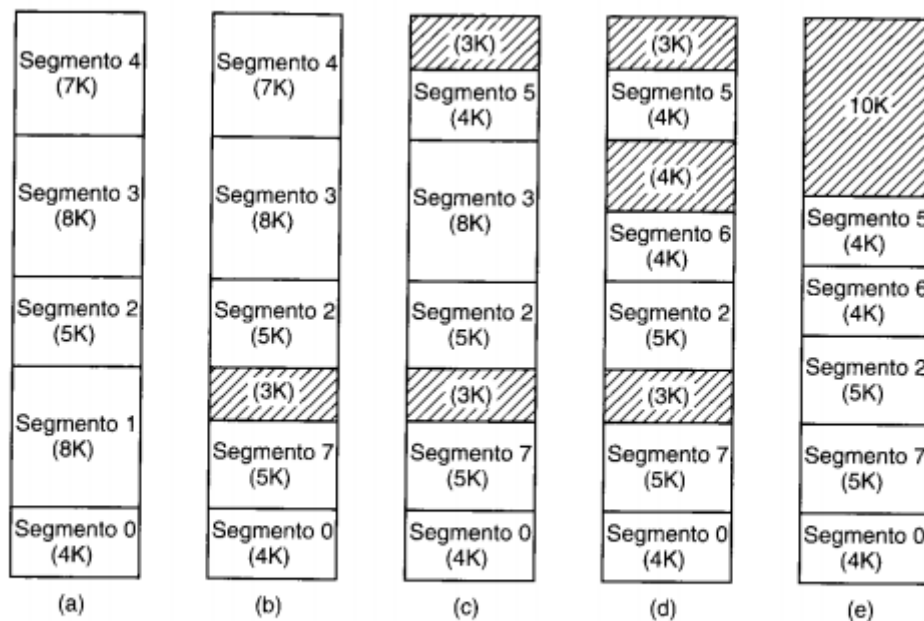


Figura 6-10. (a)-(d) Aparición de fragmentación externa. (e) Eliminación de la fragmentación externa por compactación.

Una posible solución sería reordenar los segmentos de manera que no queden espacios de memorias sin utilizar (proceso llamado **compactación**) pero este podría llegar a ser muy costoso.

La otra manera de implementar la segmentación es dividir cada segmento en páginas de tamaño fijo y paginarlas por demanda. En este esquema, algunas páginas de un

segmento podrían estar en la memoria y otras en el disco. Para paginar un segmento se requiere una tabla de páginas individual para cada segmento. Puesto que un segmento no es más que un espacio de direcciones lineal, todas las técnicas que hemos visto para la paginación se aplican a cada segmento. *La única novedad aquí es que cada segmento tiene su propia tabla de páginas.*

Uno de los primeros S.O. que combinó estas dos técnicas fue **MULTICS** (*MULTiplexed information and computing service*). Las direcciones de multics tenían dos partes: un número de segmento y una dirección dentro del segmento. Cuando se presentaba una dirección virtual al hardware, se usaba el número de segmento como índice del segmento de descriptors para localizar el descriptor del segmento accesado, como se muestra en la siguiente figura. El descriptor apuntaba a la tabla de páginas, la que permitía paginar cada segmento de la forma acostumbrada. Para mejorar el desempeño, las combinaciones segmento/página de uso más reciente se guardaban en una **memoria asociativa** de 16 entradas de hardware que permitía buscarlas rápidamente

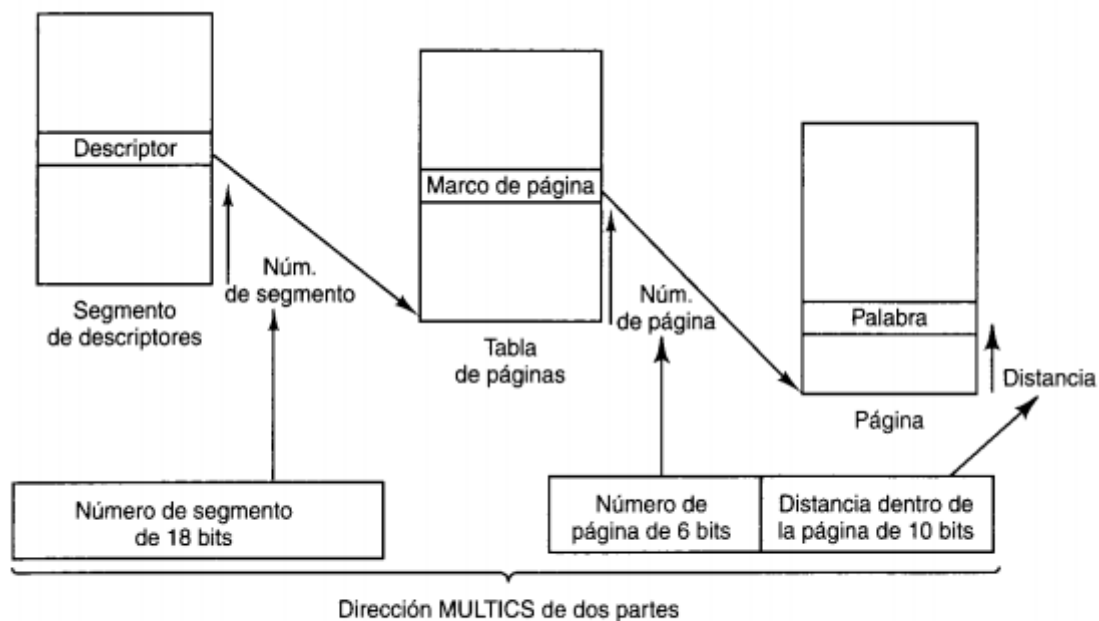


Figura 6-11. Conversión de una dirección MULTICS de dos partes en una dirección de memoria principal.

Memoria virtual en el Pentium II

El corazón de la memoria virtual del Pentium II consiste en dos tablas: la **tabla de descriptors locales (LDT)** y la **tabla de descriptors globales (GDT)**. Cada programa tiene su propia LDT, pero hay una sola GDT compartida por todos los programas de la

computadora. La LDT describe los segmentos locales de cada programa, incluidos los del código, datos, pila, etc. , mientras que la GDT describe los segmentos del sistema, incluido el sistema operativo mismo.

Cada vez que el Pentium II accede a un segmento, primero debe cargar un selector de ese segmento en uno de sus registros de segmento. Durante la ejecución, el CS contiene el selector del segmento de código, DS contiene el segmento de datos, etc. Cada selector es un número de 16 bits como se muestra en la sig. figura:

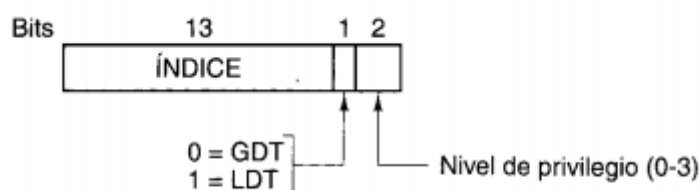


Figura 6-12. Selector de Pentium II.

Uno de los bits indica si el segmento es local o global. Otros 13 bits especifican el numero de entrada de la LDT o GDT, así que las tablas estas solo pueden contener 8K (2^{13}) descriptores de segmentos. Los otros dos bits tienen que ver con la protección y se explicaran más adelante. Cuando un selector se carga en un registro de segmento, el descriptor correspondiente se trae de la LDT o GDT y se guarda en registros internos de la MMU, para poder acceder a él rápidamente. Un descriptor consiste en 8 bytes e incluye la dirección base del segmento, su tamaño y otra información como se muestra en la figura siguiente.

El formato del selector se escogió de ingeniosamente a modo de facilitar la localización del descriptor. Primero se selección la LDT o bien la GDT, con base en el bit 2 del selector. Luego el selector se copia en un registro de borrador de la MMY y los tres bits de orden bajo se ponen en 0, con lo que el numero de selector de 13 bits se multiplica efectivamente por 8.

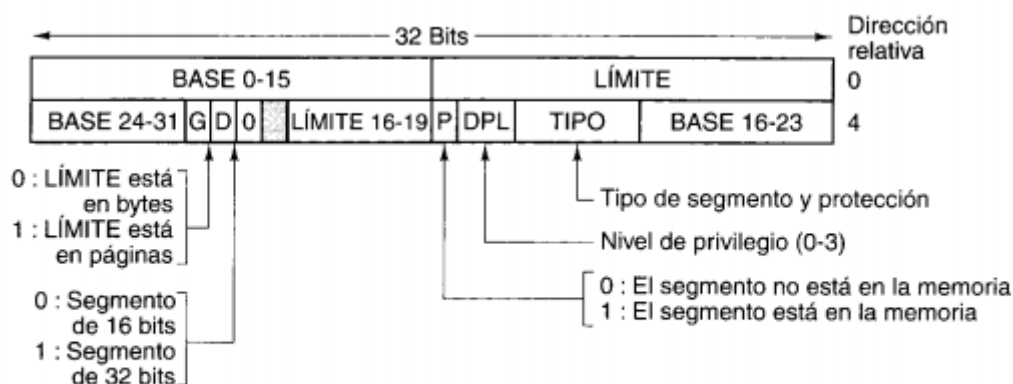


Figura 6-13. Descriptor de segmento de código de Pentium II. Los segmentos de datos son un poco diferentes.

Por último, se le suma la dirección de la tabla LDT o GDT (que se guarda en registros internos de la MMU) para dar un apuntador directo al descriptor. Por ejemplo, el selector 72 se refiere a la entrada 9 de la GDT, que se encuentra en la dirección $GDT + 72$.

Tan pronto como el hardware sabe cual registro de segmento se está usando, puede encontrar el descriptor completo que corresponde a ese selector en sus registros internos. Si el segmento no existe o no está en memoria, ocurre un error. El primer caso es un error de programación; el segundo obliga al S.O. a obtener el segmento.

Luego el sistema, verifica si la distancia rebasa el final del segmento, lo cual genera otro error. Lógicamente, debería haber un campo de 32 bits en el descriptor que diera el tamaño de segmento, pero solo cuenta con 20 bits, así que emplea un esquema distinto. Si el campo G es 0, el campo LIMITE da el tamaño exacto del segmento, hasta 1 MB; si es 1, el LIMITE da el tamaño del segmento en páginas en lugar de bytes. El tamaño de pagina del Pentium II nunca es menor que 4KB, por lo que 20 bits son suficientes para segmentos de hasta 2^{32} bytes.

Suponiendo que el segmento está en la memoria y la distancia no se sale del intervalo, el Pentium II suma el campo de 32 bits BASE del descriptor a la distancia para formar una **dirección lineal**, como se muestra en la siguiente figura. El campo BASE se descompone en tres fragmentos que se distribuyen por todo el descriptor por razones de compatibilidad con el 80286, en el que BASE solo tiene 24 bits. Así pues, el campo BASE permite que un segmento inicie en cualquier punto dentro del espacio de direcciones lineales de 32 bits.

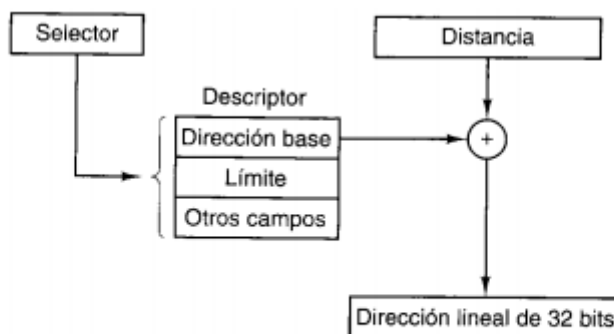


Figura 6-14. Conversión de un par (selector, distancia) en una dirección lineal.

Si se inhabilita la paginación, la dirección lineal se interpreta como la dirección física y se envía a memoria para efectuar la lectura o escritura. Por otra parte si la paginación está habilitada la dirección lineal se interpreta como una virtual y se transforma en una dirección física mediante tablas de páginas, prácticamente igual que en los anteriores ejemplos.

Cada programa en ejecución tiene un **directorio de páginas** que consta de 1024 entradas de 32 bits y se encuentra en una dirección a la que un registro global apunta. Cada entrada de este directorio apunta a una tabla de páginas que también contiene

1024 entradas de 32 bits. Las entradas de la tabla de páginas apuntan a marcos de páginas. El esquema es el siguiente:

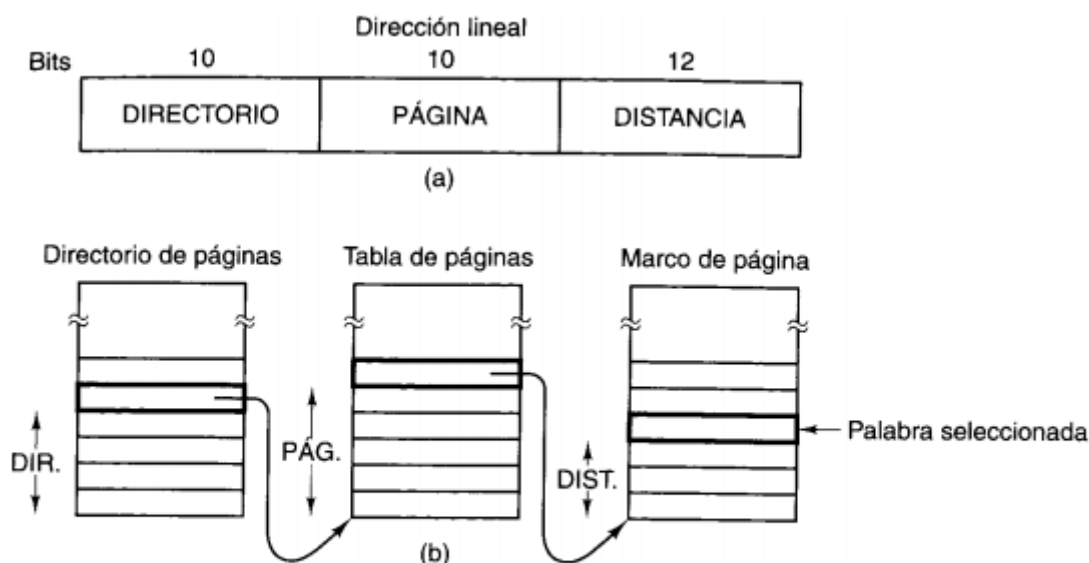


Figura 6-15. Transformación de una dirección lineal en una dirección física.

En la figura vemos una dirección lineal desglosada en tres campos, DIRECTORIO, PAGINA Y DISTANCIA. El campo DIRECTORIO se usa primero como índice del directorio de páginas para localizar un apuntador a la tabla de páginas apropiada. Luego se usa el campo PÁGINA como índice de la tabla de páginas para encontrar la dirección física del marco de página. Por último, DISTANCIA se suma a la dirección del marco de página para obtener la dirección física del byte o palabra direccionado.

Las entradas de la tabla de pagina tienen 32 bits cada una, de los cuales 20 contienen un numero de marco de pagina. Los demás bits controlan el acceso e indican si la pagina está limpia o no. El hardware ajusta estos bits que son utilizados por el S.O., los bits de protección y otros bits utilitarios.

Entrada y salida

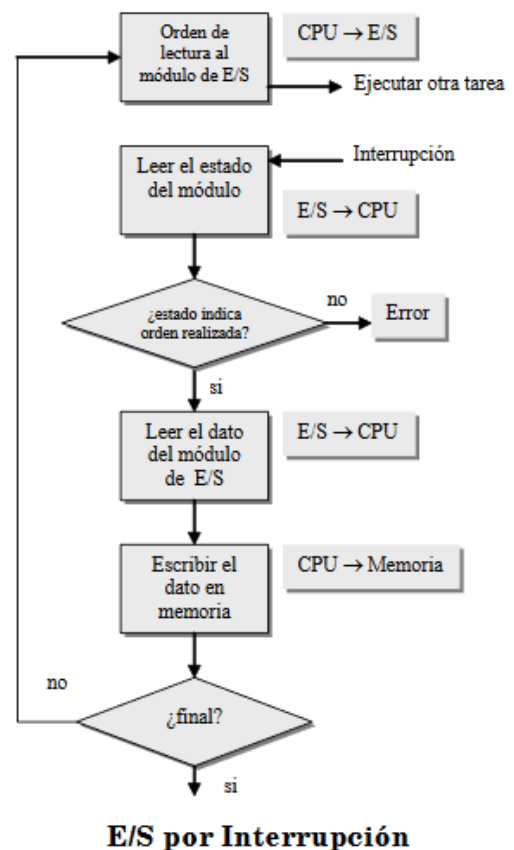
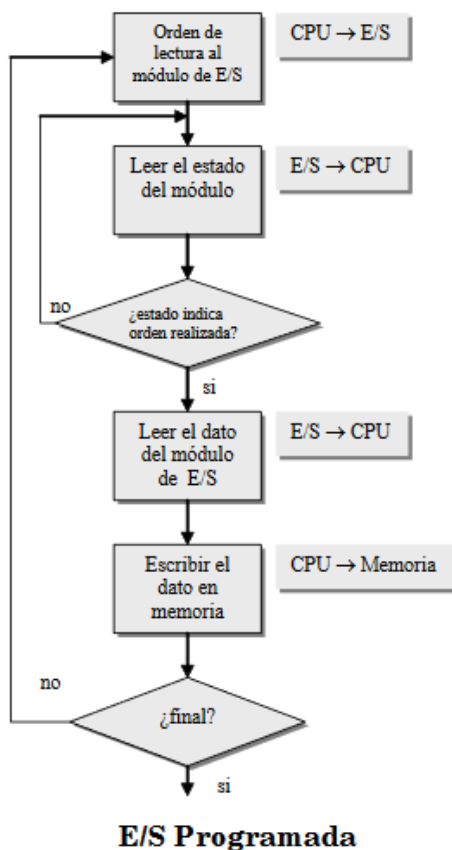
	Sin interrupciones	Usando interrupciones
Transferencia de E/S a memoria a través de la CPU	E/S Programada	E/S mediante interrupciones
Transferencia directa de E/S a memoria		Marta: Roberto camine a tu lado soñando, imaginando un futuro a tu lado, visualizando una familia juntos, pero queda en un sueño porque no

		me dijiste que tenés 8 hijos! Roberto: Déjame explicarte Marta: Me mentiste y yo como una ingenua haciendo todo por ti. Roberto: Per.... Marta: No almorzaba por pensar en ti, no comía por pensar en ti, no cenaba por pensar en ti y no dormía porque TENÍA HAMBRE! Roberto: Entonces seguí pensando en mi así no comes, gorda. Marta: Ya baje 5 kilos infeliz Roberto: Si se nota, por eso la cama se hunde cuando te acostas Marta: No te lo puedo creer por eso vete, olvida mi nombre, mi cara, mi casa y pega la vuelta Roberto: NO EXAGERES MARTA! Marta: Como me pudiste mentir con lo de tus hijos a memoria (DMA)
--	--	---

E/S Programada

Es el método de E/S más sencillo posible, que suele usarse en procesadores de bajo costo, como los sistemas incorporados o en sistemas de tiempo real. Sin embargo, presenta algunos inconvenientes:

- Pérdida de tiempo: la computadora no realiza trabajo útil en el bucle de espera.
- Impide realización de tareas periódicas.
- Dificultades para atender varios periféricos.



E/S por interrupción

El problema con la E/S programada es que el procesador tiene que esperar un tiempo considerable a que el módulo de E/S en cuestión esté preparado para recibir o transmitir datos. El procesador mientras espera, debe comprobar repetidamente el estado del módulo de E/S.

Una alternativa consiste en que el procesador, tras enviar una orden de E/S continúe realizando trabajo útil. Después, el módulo E/S interrumpirá al procesador para solicitar su servicio cuando esté preparado para intercambiar datos con él. El procesador ejecuta entonces la transferencia de datos, como antes y después continua con el procesamiento previo

Estudiaremos como funciona, primero desde el punto de vista del módulo de E/S. Para una entrada, el módulo E/S recibe una orden READ del procesador. Entonces el módulo E/S procede a leer el dato desde el periférico asociado. Una vez que el dato está en el registro de datos del módulo, el módulo envía una interrupción al procesador a través de una línea de control. Después, el módulo espera hasta que el procesador solicite su dato. Cuando ha recibido la solicitud, el módulo sitúa su dato en el bus de datos y pasa a estar preparado para otra operación E/S.

Desde el punto de vista del procesador, las acciones para una entrada son las que siguen. El procesador envía una orden READ de lectura. Entonces pasa a realizar otro trabajo. Al final de cada ciclo de instrucción, el procesador comprueba las

instrucciones. Cuando se pide la interrupción desde el modulo de E/S, el procesador guarda el contexto del programa en curso y procesa la interrupción. En este caso el procesador lee la palabra de datos del modulo E/S y la almacena en memoria. Después recupera el contexto del programa que estaba ejecutando y continúa con su ejecución.

Cuando se produce una interrupción se disparan una serie de eventos en el procesador, tanto a nivel de hardware como de software. La siguiente figura muestra una secuencia típica:

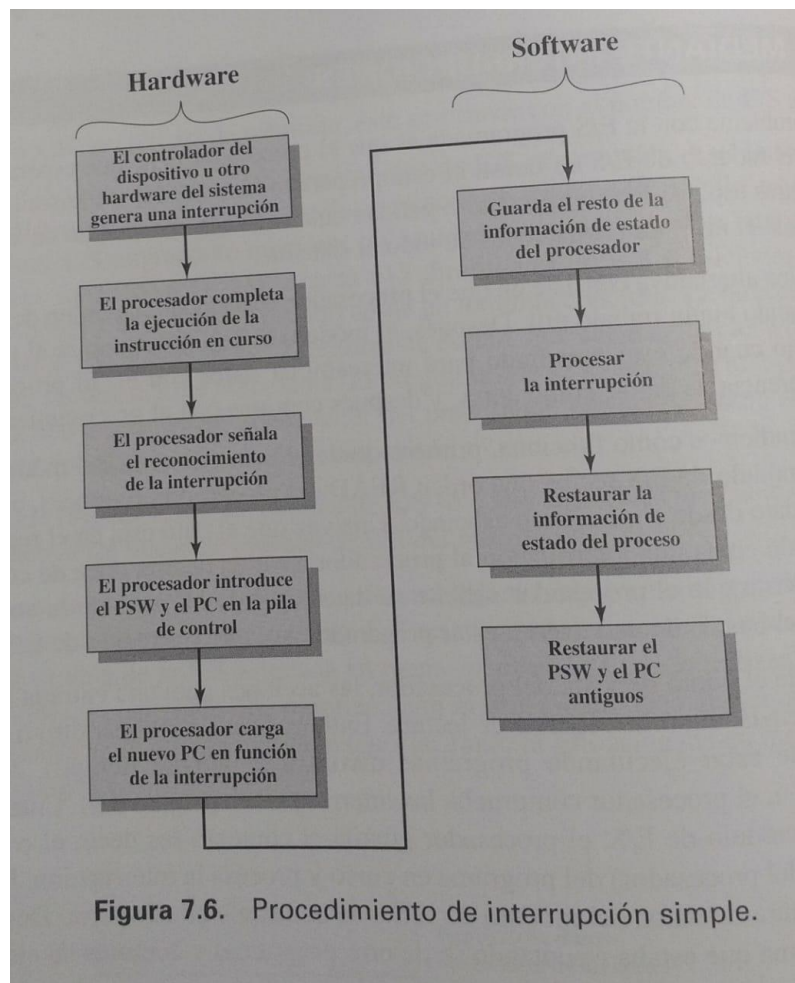


Figura 7.6. Procedimiento de interrupción simple.

La E/S con interrupciones es más eficiente que la E/S programada porque elimina las esperas innecesarias. No obstante, las E/S con interrupciones todavía consumen gran cantidad del tiempo del procesador puesto que cada palabra de datos que va desde la memoria al modulo de E/S o viceversa debe pasar a través del procesador.

Acceso directo a memoria (DMA)

Hemos añadido un número chip al sistema, un controlador de acceso directo a la memoria (**DMA**), con acceso directo al bus.

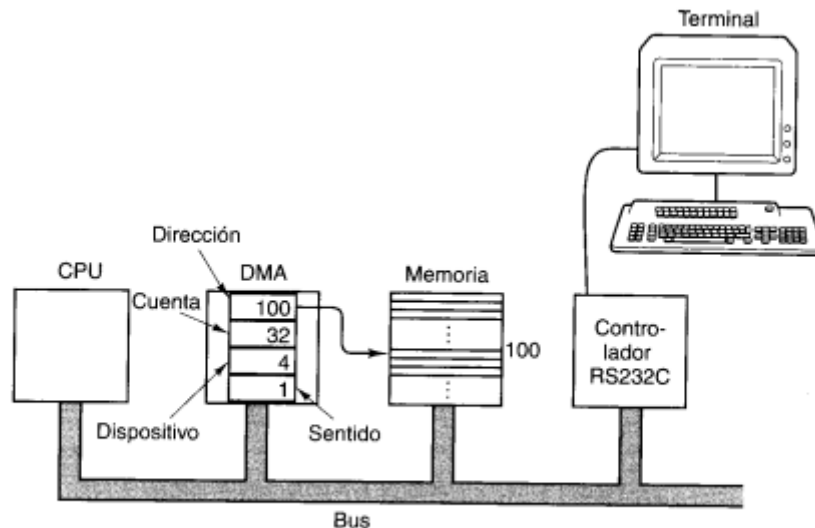


Figura 5-32. Sistema con controlador DMA.

El chip DMA contiene (al menos) cuatro registros, todos los cuales deben ser cargados por software que se ejecuta en la CPU. El primero contiene la dirección de memoria que se leerá o en la que se escribirá. El segundo contiene el número de bytes (o palabras) que se transferirán. El tercero especifica el número de dispositivos o espacio de direcciones de E/S que se usará, con lo que se especifica que dispositivo de E/S se desea. El cuarto indica si los datos se escribirán por el dispositivo E/S o se leerán de él.

Para escribir un bloque de 32 bytes de la dirección de memoria 100 en una terminal (digamos, el dispositivo 4), la CPU escribe los números 100, 32 y 4 en los primeros tres registros del DMA, y luego el código de escribir (digamos 1), en el cuarto, como se mostraba en la anterior figura. Una vez inicializado con estos valores, el controlador de DMA emite una solicitud al bus para leer el byte 100 de la memoria, del mismo modo como la CPU leería de la memoria. Una vez que obtiene el byte, el controlador de DMA, emite una solicitud de E/S al dispositivo 4, para escribir el byte en él. Una vez completada ambas operaciones, el controlador DMA incrementa en 1 su registro de dirección y decrementa en 1 su registro de cuenta. Si el registro de cuenta sigue siendo mayor que 0, se lee otro byte de la memoria y se escribe en el dispositivo.

Con el DMA, la CPU solo tiene que inicializar unos registros. Una vez hecho eso, queda libre para hacer otras cosas hasta que se completa la transferencia, momento en el cual recibe una interrupción del controlador de DMA. Si bien el DMA alivia a la CPU de gran parte de la carga de E/S, el proceso no es totalmente gratuito. Si un dispositivo de alta velocidad, como un disco, se está controlando por DMA, se requerirán muchos ciclos de bus, tanto para las referencias a la memoria como para las referencias al dispositivo. Durante estos ciclos la CPU tendrá que esperar, el proceso por el que el controlador de DMA le quita ciclos de bus a la CPU se denomina **robo de ciclos**.